

UNIVERSITÀ DEGLI STUDI DI ROMA “TOR VERGATA”

Macroarea di Ingegneria
Corso di Laurea Magistrale in Ingegneria Informatica

Prodotto matrice densa per multivettore in MPI e CUDA

Progetto di fine corso

Sistemi di Calcolo Parallelo e Applicazioni
A.A. 2025/2026

Autori

Alessandro Vassallo
0366572

Andrea Galluzzi
0365026

Codice sorgente: <https://github.com/AleVassa00/parallel-computing-project>

Versione della relazione del 12 settembre 2026

Bozza di lavoro. I marcatori in rosso segnalano i punti che dipendono da campagne di misura non ancora concluse. Per la versione di consegna, commentare `\todovisibiletrue` in `preambolo.tex`.

Indice

1	Introduzione	1
1.1	Obiettivo	1
1.2	Panoramica del progetto	1
1.3	Ripartizione del lavoro	3
2	Il nucleo di calcolo	4
2.1	Formalizzazione e conteggio delle operazioni	4
2.2	Ordine dei cicli: schema A contro schema B	4
2.3	Precisione	6
2.4	Generazione riproducibile dei dati	6
3	Architettura del software	7
3.1	Moduli	7
3.2	L'interfaccia unica del kernel locale	7
3.2.1	Ciclo di vita del kernel	8
3.2.2	Contesto opaco del backend	8
3.2.3	Configurazione delle build	9
4	Parallelizzazione MPI	10
4.1	Decomposizione a griglia bidimensionale	10
4.2	Partizionamento degli indici	10
4.3	Layout dei blocchi locali	11
4.3.1	Le tre leading dimension non sono equivalenti	11
4.4	Distribuzione iniziale dei dati	12
4.5	L'algoritmo distribuito	12
4.6	Modello di costo della comunicazione	13
5	Il nucleo di CPU	14
5.1	Specializzazione su k	14
5.1.1	Il microbenchmark che misura la specializzazione	15
5.2	Perché $KB = 32$	15
5.3	Ciò che il compilatore vede	15
5.4	Il costo nascosto: la rilettura di X	16
6	I backend CUDA	17
6.1	Struttura comune	17
6.2	<code>cuda_naive</code> : la baseline	19
6.3	<code>cuda_warp</code> : un warp per riga	19
6.3.1	Un secondo layout per X : <code>X_LAYOUT=column</code>	19
6.4	<code>cuda_warp_smem</code> : la fetta di X in shared memory	20

6.4.1	Il conflitto sui banchi e il padding a $k + 1$	21
6.4.2	Che cosa il tile cambia davvero, e che cosa no	21
6.4.3	Perché non ci sono uscite anticipate	22
6.5	<code>cuda_warp_tiled</code> : meno accumulatori per lane	22
6.6	<code>cublas</code> : il riferimento esterno	22
6.6.1	Riga contro colonna: nessuna trasposizione	23
6.6.2	L'attesa è che vada male	23
6.7	Più rank MPI sulla stessa GPU	23
7	Modello roofline	25
7.1	Roofline della GPU	25
7.1.1	Che cosa il modello non dice	27
7.2	Roofline della CPU	27
8	Metodologia sperimentale	28
8.1	Ambiente di misura	28
8.2	Che cosa entra nel cronometro	28
8.3	I tempi esclusi dalla misura ufficiale	29
8.4	Stabilità delle misure	30
8.5	Le campagne	30
8.5.1	C1 — dimensioni crescenti per rapporto $M:N$	31
8.5.2	C2 — sweep su k , un backend per riga	31
8.5.3	C3 — baseline di CPU e specializzazione contro fallback	32
8.5.4	C4 — strong scaling	32
8.5.5	C5 — weak scaling	32
8.5.6	C6 — forma della griglia a P fisso	32
8.5.7	C7 — la precisione, misurata e spiegata	32
8.6	Riproducibilità	33
9	Correttezza e validazione	34
9.1	L'oracolo seriale	34
9.2	Una tolleranza che dipende da N	34
9.3	Il test delle funzioni indice	35
9.4	La matrice di collaudo	36
10	Risultati	37
10.1	Stato della raccolta dati	37
10.2	C3 — baseline di CPU	37
10.2.1	Lettura: due regimi separati da una soglia di cache	38
10.2.2	Conseguenza pratica	39
10.2.3	Specializzazione contro fallback	40
10.3	C2 — confronto fra backend al variare di k	40
10.3.1	Sweep sul padding della shared memory	41
10.3.2	Sweep sulla dimensione del blocco	42
10.4	C1 — taglia e rapporto $M:N$	42
10.5	C6 — perché la griglia è bidimensionale	43
10.6	C4 — strong scaling	43
10.7	C5 — weak scaling	44
10.8	C7.1 — quanto rende la singola precisione	45
10.9	C7.2 — perché: registri, spill e shared memory	46
10.10	Il costo dei dati: trasferimenti e preprocessing	46
10.11	Sintesi	47

11 Conclusioni	48
11.1 Che cosa è stato realizzato	48
11.2 Che cosa dicono i dati finora	48
11.3 Lavori futuri	49
A Interfaccia a riga di comando e formato dei dati	50
A.1 Opzioni del driver	50
A.2 Colonne del CSV	50
A.3 Opzioni di build	51
B Riproduzione dei risultati	53
B.1 Build	53
B.2 Validazione	53
B.3 Esecuzione singola	54
B.4 Campagne	54
B.5 Unione dei CSV di una campagna	54
B.6 Note operative sul server	54
B.7 Compilazione di questa relazione	55

Capitolo 1

Introduzione

1.1 Obiettivo

Il progetto realizza un nucleo di calcolo per il prodotto fra una matrice densa e un multivettore,

$$Y \leftarrow AX, \quad A \in \mathbb{R}^{M \times N}, \quad X \in \mathbb{R}^{N \times k}, \quad Y \in \mathbb{R}^{M \times k}, \quad (1.1)$$

con k *piccolo*: il multivettore è una matrice densa alta e strettissima. Il codice funziona per k generico ed è collaudato sui cinque valori richiesti dalla traccia, $k \in \{3, 6, 8, 20, 32\}$.

La parallelizzazione è ibrida a due livelli. MPI costituisce il livello esterno, responsabile della distribuzione dei dati su una griglia di processi bidimensionale e della comunicazione fra processi; la parallelizzazione interna al singolo processo è affidata a CUDA, l'opzione scelta fra le due ammesse dalla traccia. I due livelli non sono alternativi ma sovrapposti: ogni processo MPI possiede un blocco di A e delega il calcolo del proprio contributo parziale a un *backend* locale, che può essere un kernel CUDA oppure un nucleo di CPU.

1.2 Panoramica del progetto

Dimensioni e geometria scelte a runtime. Nessuna dimensione del problema è nota a tempo di compilazione: M , N , k e l'estensione $P_r \times P_c$ della griglia sono opzioni da linea di comando (capitolo A), validate prima di qualunque allocazione. L'unica eccezione, deliberata, riguarda k : i cinque valori della traccia hanno un percorso *specializzato* a tempo di compilazione, perché solo così i k accumulatori possono restare in registro (sezione 5.1); ogni altro k attraversa un percorso generico, che resta corretto ed è misurato a parte.

Distribuzione bidimensionale e gestione degli indici. I processi sono organizzati in una griglia cartesiana costruita con `MPI_Cart_create`, da cui `MPI_Cart_sub` ricava i comunicatori di riga e di colonna usati dalle collettive (sezione 4.1). A è distribuita a blocchi lungo entrambe le dimensioni: il processo di coordinate (i, j) detiene un blocco di $m_{\text{loc}} \times n_{\text{loc}}$ elementi.

Coerentemente con quanto consentito dalla traccia, i multivettori non sono distribuiti sull'intera griglia: X risiede sulla sola riga 0 di processi e viene propagata lungo ogni colonna con un `MPI_Bcast`. I contributi parziali di Y vengono sommati lungo ogni riga della griglia con un `MPI_Reduce` e raccolti sulla sola colonna 0. Ogni invocazione del prodotto costa quindi due collettive, entrambe su sottocomunicatori di dimensione ridotta, con un volume proporzionale a k .

Disaccoppiamento fra comunicazione e calcolo locale. Il livello MPI non conosce il backend con cui viene eseguito il calcolo locale: vede soltanto l'interfaccia `local_gemm` di `src/kernel/kernel.h` (sezione 3.2), a contesto opaco, con le tre operazioni di creazione, esecuzione e rilascio. Il ciclo di vita a tre fasi (sezione 3.2.1) è la scelta con l'effetto più ampio sull'intero progetto: A non cambia mai fra un'invocazione e l'altra, quindi tutto ciò che la riguarda avviene una volta sola in `local_gemm_create` e resta fuori dal cronometro per costruzione, non per sottrazione a posteriori. La stessa interfaccia è agnostica rispetto alla collocazione della memoria, host o device, il che lascia aperta la strada a un MPI CUDA-aware senza modifiche al codice di distribuzione (sezione 11.3).

Backend di calcolo locale. Sul lato GPU il progetto contiene quattro kernel e un riferimento esterno (capitolo 6), concepiti come una *sequenza* in cui ogni versione cambia una sola cosa rispetto alla precedente:

- `cuda_naive` (un thread per elemento di Y , la baseline),
- `cuda_warp` (un warp per riga di Y , k accumulatori per lane e riduzione con `__shfl_down_sync`),
- `cuda_warp_smem` (la fetta di X passa da un tile in shared memory, con padding sui banchi regolabile da `SMEM_PAD`),
- `cuda_warp_tiled` (un warp per tile di colonne).

Come accennato in precedenza, `cublas` non è una proposta, ma viene sfruttato come riferimento con il quale confrontare le performance dei vari kernel CUDA.

Sul lato CPU, troviamo tre proposte: `scheme_a` è l'implementazione di base, con due ottimizzazioni di cache `scheme_a_jblock` e `scheme_a_jblock_rb` (capitolo 5).

Tutto il codice usa un unico tipo `scalar_t`, pari a `double` per default e `float` con `PREC=float`: il confronto fra le due precisioni è un flag di compilazione sullo stesso sorgente (sezione 2.3), il modello roofline ne prevede l'effetto (capitolo 7).

Preprocessamento e collaudo funzionale. Le matrici sono generate, non lette da file, con un generatore senza stato funzione dei soli indici globali e del seme (sezione 2.4): ogni processo può produrre direttamente il proprio blocco (`-a-mode local`, `-x-mode local`), oppure il rank 0 materializza l'intera matrice e la distribuisce: A con datatype `MPI_Type_vector`, X con `MPI_Scatterv` sulla riga 0 (sezione 4.4). Generazione e distribuzione compongono una fase di preprocessing e per questo motivo sono state escluse dal tempo ufficiale, ma vengono comunque cronometrate, riportate e discusse a parte.

La correttezza è stata verificata con `-check` contro un'implementazione seriale minimale usata come oracolo (capitolo 9): il rank 0 raccoglie Y distribuita, ricostruisce A e X dal generatore, e confronta in norma L_2 relativa con una tolleranza che cresce come $\varepsilon\sqrt{N}$, coerente con la non associatività dell'aritmetica in virgola mobile.

Misura delle prestazioni. Le misure sono state condotte sul server di dipartimento: un nodo a doppio socket con una singola GPU Quadro RTX 5000 (Turing, `sm_75`), raggiungibile solo dai nodi di calcolo attraverso lo scheduler e non dal frontend (capitolo 8).

La metrica ufficiale è quella imposta dalla traccia, $\text{FLOPS} = 2MNk/T$ con T tempo medio per invocazione su esecuzioni ripetute. I tempi dei kernel sono rilevati con `cudaEvent`, e la

fase locale su GPU è scomposta per intero in tempo *host-to-device* (H2D) di X , tempo di kernel, tempo *device-to-host* (D2H) di Y e overhead del runtime (sezione 8.3).

Le campagne sono descritte da file `.conf` e lanciate da un unico runner (sezione 8.5). Un aspetto dell'ambiente va enunciato prima dei numeri: senza MPS, più rank MPI sulla stessa GPU si alternano sulla scheda, per cui con $P > 1$ il tempo di kernel è un throughput aggregato e non il tempo di una GPU dedicata. Le campagne di caratterizzazione del kernel sono perciò a rank singolo (sezione 6.7).

1.3 Ripartizione del lavoro

Da completare con i dati della campagna

La traccia richiede che un elaborato di gruppo indichi la suddivisione del lavoro fra i componenti. Compilare la tabella seguente prima della consegna.

Attività	Alessandro Vassallo	Andrea Galluzzi
Infrastruttura MPI, griglia, indici e distribuzione	–	–
Nucleo di CPU e specializzazioni su k	–	–
Backend CUDA e riferimento cuBLAS	–	–
Driver di misura, cronometri e CSV	–	–
Campagne sperimentali e analisi	–	–
Validazione e test	–	–
Relazione	–	–

Tabella 1.1: Ripartizione del lavoro. **[DA FARE: compilare]**

Capitolo 2

Il nucleo di calcolo

2.1 Formalizzazione e conteggio delle operazioni

Il nucleo calcola, per ogni riga i di Y e ogni colonna c del multivettore:

$$Y[i][c] = \sum_{j=0}^{N-1} A[i][j] X[j][c], \quad 0 \leq i < M, \quad 0 \leq c < k. \quad (2.1)$$

Ogni elemento di Y richiede N moltiplicazioni e $(N - 1)$ addizioni. Il conteggio convenzionale, e quello imposto dalla traccia, è di 2 FLOP per termine:

$$\text{FLOP}(M, N, k) = 2 M N k, \quad \text{GFLOPS} = \frac{2 M N k}{T \cdot 10^9}. \quad (2.2)$$

dove T è il tempo *medio* per invocazione, ottenuto ripetendo il nucleo più volte sulla stessa matrice. Che cosa esattamente entri in T è una scelta con conseguenze rilevanti, ed è trattata in maniera dettagliata in sezione 8.2.

La matrice A è memorizzata in formato row-major con leading dimension `lda`, che può essere maggiore del numero di colonne effettivamente possedute dal processo. Ciò permette di introdurre padding tra righe consecutive e di ricevere direttamente blocchi non contigui mediante datatype MPI derivati. L'interfaccia del kernel espone analogamente `ldx` e `ldy`; nel percorso distribuito ordinario, tuttavia, X e Y sono mantenute compatte, con `ldx` = `ldy` = `k`.

2.2 Ordine dei cicli: schema A contro schema B

L'implementazione dell'equazione 2.1 ammette più ordini di annidamento dei tre cicli, che sono matematicamente equivalenti, ma non lo sono prestazionalmente. La scelta dell'ordine dei cicli è il primo punto in cui il progetto si differenzia da una trascrizione letterale della definizione, e il motivo è l'intensità aritmetica del nucleo. I due schemi candidati dell'ordine dei cicli per l'implementazione dell'equazione, sono:

Schema A ($i \rightarrow j \rightarrow c$)

```

for i:
  acc[0..k) = 0
  for j:
    a = A[i][j]
    for c:
      acc[c] += a * X[j][c]
  Y[i][0..k) = acc[0..k)

```

Schema B ($i \rightarrow c \rightarrow j$)

```

for i:
  for c:
    s = 0
    for j:
      s += A[i][j] * X[j][c]
    Y[i][c] = s

```

Ogni schema ha le proprie caratteristiche e vantaggi. Lo schema B è la trascrizione letterale della definizione e corrisponde esattamente all'implementazione di k prodotti matrice-vettore indipendenti: la riga i di A viene riletta una volta per ciascuna colonna c , cioè k volte. Lo schema A invece legge $A[i][j]$ **una sola volta** e lo riusa immediatamente per tutti i k accumulatori, che vivono in registro. Lo schema A è adottato dai backend CPU come organizzazione di base del calcolo e costituisce inoltre un riferimento per le successive ottimizzazioni orientate al riuso di A . I backend CUDA introducono invece differenti strategie di mapping del lavoro sui thread e sui warp, descritte nel capitolo dedicato.

Il vantaggio si quantifica con l'*intensità aritmetica*, cioè il rapporto fra FLOP eseguiti e byte che attraversano il livello di memoria considerato. Il solo traffico sulla matrice A conta $\Theta(MN)$ e domina, mentre X e Y sono $\Theta(Nk)$ e $\Theta(Mk)$ con k piccolo. Detta s la dimensione in byte dello scalare, che varia a seconda della precisione, l'intensità aritmetica dei due schemi è:

$$I_A = \frac{2MNk}{MNs} = \frac{2k}{s}, \quad I_B = \frac{2MNk}{kMNs} = \frac{2}{s}. \quad (2.3)$$

In doppia precisione FP64 ($s = 8$ byte) questo significa che si ha $I_A = k/4$ FLOP/B contro $I_B = 1/4$ FLOP/B: lo schema A guadagna esattamente un fattore k , e per $k = 32$ porta l'intensità da 0,25 a 8 FLOP/B. È questo salto che sposta il nucleo da un regime saldamente memory-bound a un regime in cui, almeno sulla GPU in FP64, il limite può diventare il calcolo quindi un regime compute-bound, come discusso nel capitolo dedicato al modello roofline (capitolo 7).

Vale la pena osservare che $I_A = 2k/s$ è il *massimo* ottenibile leggendo A una volta sola: nessuna riorganizzazione ulteriore può migliorarlo, perché MN elementi di A vanno comunque letti almeno una volta e i FLOP sono fissati. Perciò il resto del lavoro di ottimizzazione, sia su CPU sia su GPU, riguarda quindi il traffico su X e la capacità di tenere i k accumulatori in registro e non l'intensità rispetto ad A , che è già al suo limite.

Un'ultima proprietà dello schema A che conta sull'hardware reale: entrambi gli stream di lettura sono a passo unitario. La riga di A è contigua per costruzione, e la riga di X lo è perché X è row-major con le k colonne adiacenti. Nessuno dei due accessi ha stride, quindi i prefetcher hardware lavorano nelle condizioni migliori e, sulla GPU, gli accessi delle lane di uno stesso warp sono coalescibili.

2.3 Precisione

Tutto il codice usa un unico tipo `scalar_t`, definito in `src/common/scalar.h`, che vale `double` per default e `float` compilando con `PREC=float`. Il confronto FP64/FP32 è quindi un flag di compilazione e non una seconda copia dei sorgenti: le due build eseguono esattamente lo stesso codice sorgente, che è la sola condizione in cui il confronto misura la precisione e non due implementazioni diverse. Anche il tipo MPI segue lo scalare, tramite la macro `SCALAR_MPI_TYPE`.

La scelta della precisione non è neutra rispetto al modello roofline (capitolo 7): dimezzando s raddoppia l'intensità aritmetica della equazione 2.3, ma su una GPU consumer o professionale di classe Turing il picco FP32 è 32 volte il picco FP64. I due effetti vanno nella stessa direzione e spostano il nucleo, in singola precisione, saldamente nella regione memory-bound: la discussione quantitativa è in sezione 7.1.

2.4 Generazione riproducibile dei dati

Le matrici di test sono generate, non lette da file. La traccia lo consente esplicitamente e chiede in cambio che la generazione sia *ripetibile*.

Il generatore è `splitmix64`, un mixer a 64 bit **senza stato**:

$$A[i][j] = g(\text{seed}, \Sigma_A, i \cdot N + j), \quad X[j][c] = g(\text{seed}, \Sigma_X, j \cdot k + c), \quad (2.4)$$

dove Σ_A e Σ_X sono due identificatori di stream distinti e la chiave è l'**indice lineare globale** dell'elemento. Il valore prodotto dipende dunque *solo* dalla posizione globale, mai dal processo che lo calcola né dall'ordine in cui viene generato. Le conseguenze sono diverse, e sono tutte usate dal progetto:

1. ogni processo può generare **direttamente** il proprio blocco locale, senza che esista mai una copia globale della matrice.
2. la stessa matrice globale si ottiene con *qualunque* griglia di processi e con qualunque modalità di inizializzazione, quindi due configurazioni diverse sono confrontabili elemento per elemento;
3. l'oracolo seriale di capitolo 9 può ricostruire da sé la porzione che gli serve, senza ricevere dati dagli altri processi.

I valori sono distribuiti in $[-1, 1)$ a media nulla.

Capitolo 3

Architettura del software

3.1 Moduli

Il codice è organizzato in moduli distinti in base alle rispettive responsabilità. Ogni sottocartella di `src/` raccoglie le funzionalità relative a uno specifico aspetto dell'applicazione.

Tabella 3.1: Struttura di `src/`.

Modulo	Responsabilità
<code>common/</code>	definizione del tipo scalare (<code>scalar_t</code>), configurabile come <code>float</code> o <code>double</code> , tolleranza di validazione, utility di terminazione e cronometro
<code>index/</code>	partizionamento a blocchi degli indici globali: dimensione, offset e proprietario di ogni blocco
<code>gen/</code>	generatore riproducibile senza stato (sezione 2.4)
<code>serial/</code>	implementazione seriale minimale, usata come oracolo
<code>kernel/</code>	<code>local_gemm</code> : un file per implementazione, <code>.c</code> o <code>.cu</code> , tutte dietro la stessa interfaccia
<code>mpi/</code>	griglia cartesiana, layout dei blocchi, distribuzione dei dati, algoritmo distribuito $Y = AX$
<code>bench/</code>	driver di misura e validazione, parsing delle opzioni, output CSV

Il tipo `scalar_t` permette di utilizzare la stessa base di codice sia in singola precisione (`float`, FP32) sia in doppia precisione (`double`, FP64); la precisione viene selezionata a tempo di compilazione.

Il modulo `mpi/` non dipende direttamente da uno specifico backend di calcolo: per l'esecuzione del prodotto locale utilizza esclusivamente l'interfaccia comune definita in `kernel/kernel.h`. Di conseguenza, l'algoritmo MPI rimane invariato sia utilizzando un'implementazione CPU sia utilizzando uno dei backend CUDA. La scelta del backend avviene quindi in fase di compilazione, senza richiedere modifiche al codice che gestisce la distribuzione dei dati e le comunicazioni MPI.

3.2 L'interfaccia unica del kernel locale

Ogni backend implementa tre operazioni fondamentali per la gestione e l'esecuzione del kernel locale, oltre ad alcune funzioni ausiliarie utilizzate per la raccolta delle metriche sperimentali.

```
/* esecuzione */
local_gemm_t *local_gemm_create(int m, int n, int k,
```

```

        const scalar_t *A_loc,
        int lda, int ldx, int ldy);
void local_gemm(local_gemm_t *ctx,
        const scalar_t *RESTRICT X, int ldx,
        scalar_t *RESTRICT Y, int ldy);
void local_gemm_destroy(local_gemm_t *ctx);

```

Listato 3.1: Interfaccia di `src/kernel/kernel.h`.

L'interfaccia segue un ciclo di vita composto da tre fasi rappresentate da altrettante funzioni che definiscono il ciclo di vita del kernel locale: `local_gemm_create` ne inizializza il contesto, `local_gemm` esegue il calcolo e `local_gemm_destroy` rilascia le risorse associate.

Inoltre l'interfaccia espone alcune funzioni di introspezione utilizzate dal driver di benchmark per raccogliere separatamente tempi di calcolo, trasferimenti, costi di inizializzazione e parametri di esecuzione del backend. Tali metriche vengono descritte nella sezione dedicata alla metodologia sperimentale.

3.2.1 Ciclo di vita del kernel

La separazione tra `local_gemm_create`, `local_gemm` e `local_gemm_destroy` deriva dal diverso ciclo di vita dei dati e delle risorse coinvolte nel calcolo.

La matrice A rimane invariata durante tutte le ripetizioni, mentre X e Y sono coinvolte in ogni esecuzione del prodotto locale. Questa differenza permette di separare le operazioni eseguite una sola volta da quelle necessarie a ogni invocazione.

1. **Inizializzazione (`local_gemm_create`).** Viene creato il contesto del backend e vengono predisposte le risorse riutilizzabili durante le successive esecuzioni.

Nei backend CPU vengono memorizzate le informazioni necessarie all'esecuzione, tra cui le dimensioni del problema e il riferimento al blocco locale di A .

Nei backend CUDA vengono inoltre allocati i buffer sul device e A viene trasferita una sola volta nella memoria della GPU.

2. **Esecuzione (`local_gemm`).** Questa fase viene ripetuta per ogni invocazione del prodotto locale. La matrice X , precedentemente distribuita tra i processi tramite `MPI_Bcast`, viene utilizzata insieme al blocco locale di A per produrre il contributo locale a Y .

Nei backend CUDA vengono riutilizzati i buffer allocati durante l'inizializzazione: la matrice X viene trasferita sul device, viene eseguito il kernel e il risultato Y viene riportato in memoria host.

I contributi prodotti dai processi vengono successivamente combinati tramite `MPI_Reduce`.

3. **Finalizzazione (`local_gemm_destroy`).** Al termine delle ripetizioni vengono rilasciate le risorse associate al contesto, comprese le eventuali allocazioni sulla GPU.

Questa organizzazione evita di ripetere a ogni invocazione le operazioni associate ad A e separa il costo di inizializzazione dal costo delle successive esecuzioni del prodotto locale.

3.2.2 Contesto opaco del backend

Lo stato specifico di ciascun backend è mantenuto all'interno del tipo opaco `local_gemm_t`. La struttura concreta è definita separatamente da ogni implementazione e può quindi contenere informazioni differenti senza modificare l'interfaccia comune.

Ad esempio, un backend CPU può memorizzare le dimensioni del problema e il riferimento ad A , mentre un backend CUDA mantiene anche i puntatori alla memoria del device e le risorse necessarie all'esecuzione. I dettagli interni del backend rimangono così nascosti al resto dell'applicazione.

3.2.3 Configurazione delle build

Le principali scelte sperimentali, tra cui backend di calcolo, precisione numerica e alcuni parametri dei kernel CUDA, vengono selezionate a tempo di compilazione tramite il **Makefile**.

Le principali varianti di configurazione utilizzate nelle campagne sperimentali vengono associate a binari distinti, evitando che build differenti si sovrascrivano e rendendo immediatamente identificabile la configurazione utilizzata.

Capitolo 4

Parallelizzazione MPI

4.1 Decomposizione a griglia bidimensionale

I P processi sono organizzati in una griglia cartesiana $P_r \times P_c = P$, creata con `MPI_Cart_create` e con `reorder = 1`. La forma può essere scelta dall'utente con i flag `-pr` e `-pc`, ma in assenza di indicazione, `grid_default_shape` prende la fattorizzazione più quadrata di P (se P è primo, $1 \times P$).

Tramite `MPI_Cart_sub` si ricavano due sotto-comunicatori:

- `row_comm`, che raccoglie i processi di una stessa *riga* della griglia;
- `col_comm`, che raccoglie quelli di una stessa *colonna*.

Sono i due assi lungo cui viaggiano le uniche due collettive del cammino cronometrato.

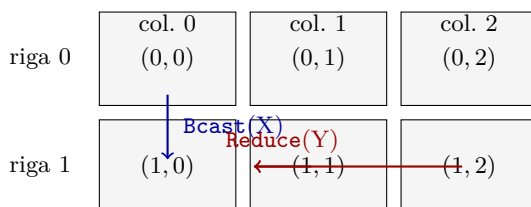


Figura 4.1: Esempio di Griglia 2×3 . X scende lungo le colonne (`col_comm`), i contributi parziali di Y si sommano lungo le righe (`row_comm`) verso la colonna 0.

Il codice fa affidamento su una proprietà dei sotto-comunicatori: in un comunicatore con una sola dimensione residua i rank sono ordinati per coordinata crescente. Ciò significa che il rank 0 di `col_comm` è il processo di riga 0 (il *responsabile* del broadcast di X , cioè il root del sotto-comunicatore) e il rank 0 di `row_comm` è il processo di colonna 0 (il *responsabile* della reduce di Y , cioè il root del sotto-comunicatore).

La proprietà viene comunque verificata in `grid_create` confrontando i rank con le coordinate cartesiane: dopo `reorder = 1` i rank possono differire da quelli di `MPI_COMM_WORLD`, e appoggiarsi a un'ipotesi non controllata darebbe risultati sbagliati in silenzio.

4.2 Partizionamento degli indici

La partizione è a blocchi, con il resto distribuito sulle prime parti. Per n elementi su p parti, la parte i ha dimensione e offset:

$$\text{size}(n, p, i) = \left\lfloor \frac{n}{p} \right\rfloor + \begin{cases} 1 & i < (n \bmod p) \\ 0 & \text{altrimenti} \end{cases} \quad \text{start}(n, p, i) = i \left\lfloor \frac{n}{p} \right\rfloor + \min(i, n \bmod p). \quad (4.1)$$

La funzione inversa $\text{owner}(n, p, g)$ restituisce la parte che possiede l'indice globale g senza scorrere le parti. Le tre funzioni sono in `src/index/index.c` e sono indipendenti da MPI.

Lo sbilanciamento massimo introdotto è di un elemento per dimensione, cioè al più una riga e una colonna di differenza fra due blocchi locali.

4.3 Layout dei blocchi locali

La matrice A è tagliata lungo entrambe le dimensioni, X e Y lungo una sola:

$$A_{\text{loc}} : m_{\text{loc}} \times n_{\text{loc}}, \quad X_{\text{loc}} : n_{\text{loc}} \times k, \quad Y_{\text{loc}} : m_{\text{loc}} \times k, \quad (4.2)$$

con $m_{\text{loc}} = \text{size}(M, P_r, \text{my_row})$ e $n_{\text{loc}} = \text{size}(N, P_c, \text{my_col})$.

Prima del broadcast, X_{loc} contiene dati significativi soltanto sui processi della riga 0. Dopo il prodotto locale, ogni processo possiede un contributo parziale a Y ; la `MPI_Reduce` lungo `row_comm` produce il blocco finale di Y sui processi della colonna 0.

Il multivettore non viene mai tagliato lungo k . Due ragioni:

- k è piccolo, e frammentarlo produrrebbe messaggi dominati dalla latenza invece che dalla banda;
- una divisione ridurrebbe il riuso di A in registro (sezione 2.2), che viene proprio da k .

Invariante critico

Il numero di righe locali di X deve venire dalla *stessa* partizione con cui si tagliano le colonne di A , e il numero di righe locali di Y dalla stessa con cui si tagliano le righe di A . Qui l'invariante è garantito *per costruzione*: esistono soltanto m_{loc} e n_{loc} , calcolati una volta sola in `layout_init`, e non due coppie di valori da tenere allineate. La struttura `layout_t` è il punto unico di verità sulla forma dei blocchi.

4.3.1 Le tre leading dimension non sono equivalenti

`lda` è l'unica leading dimension che può essere maggiore della larghezza del blocco. `ldx` e `ldy` valgono *sempre* k , e il codice lo impone con un controllo esplicito in `mpi_matmul`.

La ragione è che X e Y attraversano una collettiva: `MPI_Bcast` e `MPI_Reduce` usano conteggi contigui ($n_{\text{loc}} \cdot k$ e $m_{\text{loc}} \cdot k$ elementi): descrivere uno stride richiederebbe di costruire un `MPI_Type_vector`, committarlo e liberarlo a *ogni invocazione*, cioè tre chiamate al runtime MPI dentro la regione cronometrata ed è esattamente ciò che la misura non deve contenere.

Al contrario A non attraversa nessuna collettiva: il suo padding resta un fatto locale al kernel e può quindi essere studiato.

Il target `make check-padding` compila con `lda = nloc + 8` e verifica che tutti i backend onorino la leading dimension.

4.4 Distribuzione iniziale dei dati

Le modalità di inizializzazione di A e X sono indipendenti (`-a-mode`, `-x-mode`), e tutte e quattro le combinazioni sono valide. Entrambe producono gli stessi elementi globali a parità di seed, per la proprietà del generatore discussa in sezione 2.4.

-a-mode global. Il grid rank 0 materializza A completa e la distribuisce. È il caso interessante dal punto di vista MPI: i sottoblocchi 2D **non sono contigui** nella matrice row-major globale. Il mittente descrive il blocco da inviare con un `MPI_Type_vector` di m_{loc} blocchi da n_{loc} elementi e stride N . Il destinatario ne descrive uno con stride `lda`, che può essere diverso. **È il caso generale - due layout diversi sui due lati della stessa comunicazione - e non richiede nessun buffer intermedio né nessuna copia manuale.**

-a-mode local. Ogni processo genera direttamente il proprio blocco 2D di A dagli indici globali. In questo modo non è necessario materializzare una copia globale aggiuntiva della matrice su un singolo processo.

-x-mode global Il grid rank 0 materializza la X compatta $N \times k$ e un `MPI_Scatterv` sulla sola riga 0 ne distribuisce i blocchi di righe: entrambi i lati sono contigui, perché `ldx = k` ovunque, quindi non serve nessun datatype derivato.

-x-mode local I processi della riga 0 generano direttamente le proprie righe di X . Il successivo broadcast lungo `col_comm` è identico alla modalità `global`.

Generazione e distribuzione iniziale fanno parte del **preprocessing**, perciò sono escluse dai tempi ufficiali. La modalità `global` rappresenta uno scenario in cui l'input è inizialmente centralizzato.

4.5 L'algoritmo distribuito

Un'invocazione del prodotto esegue tre passi:

```
MPI_Bcast(X_loc, n_loc*k, SCALAR_MPI_TYPE, 0, grid->col_comm);

local_gemm(local_gemm_context, X_loc, layout->ldx, Y_loc_part, layout->ldy);

MPI_Reduce(Y_loc_part, Y_row_col0, m_loc*k, SCALAR_MPI_TYPE,
           MPI_SUM, 0, grid->row_comm);
```

Listato 4.1: Il corpo di `mpi_matmul`, senza i cronometri.

1. **Broadcast di X lungo `col_comm`:** tutti i processi di una colonna della griglia lavorano sulle stesse colonne di A , quindi servono loro le stesse righe di X . Il root è il processo di riga 0, l'unico che possiede inizialmente quella fetta, ne è il *responsabile*.
2. **Prodotto locale:** il risultato è *parziale*: il processo in posizione (r, c) calcola il contributo delle sole colonne $[\text{col}_0, \text{col}_0 + n_{\text{loc}})$ di A alle righe $[\text{row}_0, \text{row}_0 + m_{\text{loc}})$ di Y (dove row_0 e col_0 sono gli offset globali del blocco locale).
3. **Reduce lungo `row_comm`:** i P_c contributi parziali della stessa banda di righe vivono sui processi di una stessa riga della griglia: sommandoli tra loro si ottiene Y . Il `count` è $m_{\text{loc}} \cdot k$ (e non 1) dato che la riduzione agisce lungo la dimensione dei processi e preserva la posizione, producendo dunque $m_{\text{loc}} \cdot k$ somme indipendenti.

Il root è il processo di colonna 0, che è già il *responsabile* designato di quella porzione di Y : nessuna ridistribuzione finale è necessaria.

4.6 Modello di costo della comunicazione

Nel modello α - β , un messaggio di n byte costa $\alpha + \beta n$. Con algoritmi ad albero per le due collettive e s byte per scalare, il costo di comunicazione per invocazione è

$$T_{\text{comm}} \approx \underbrace{\log_2(P_r) \left(\alpha + \beta s k \frac{N}{P_c} \right)}_{\text{Bcast di } X} + \underbrace{\log_2(P_c) \left(\alpha + (\beta s + \gamma) k \frac{M}{P_r} \right)}_{\text{Reduce di } Y}, \quad (4.3)$$

dove γ è il costo per elemento della somma eseguita dalla reduce.

Il termine che decide la forma ottima della griglia è il volume:

$$V(P_r, P_c) = k \left(\frac{N}{P_c} + \frac{M}{P_r} \right), \quad P_r P_c = P. \quad (4.4)$$

Minimizzando con il vincolo $P_r P_c = P$ si ottiene

$$P_r^* = \sqrt{P \frac{M}{N}}, \quad P_c^* = \sqrt{P \frac{N}{M}}, \quad V^* = \frac{2k\sqrt{MN}}{\sqrt{P}}. \quad (4.5)$$

Due letture, entrambe verificate sperimentalmente in capitolo 10:

- **Perché 2D e non 1D.** Le due decomposizioni monodimensionali sono i casi degeneri $P_r = 1$ (volume kM , nessun broadcast ma reduce su tutti i P processi) e $P_c = 1$ (volume kN , nessuna reduce ma broadcast su tutti i P). In entrambi il volume *non decresce* con P , mentre la griglia 2D lo fa scendere come $1/\sqrt{P}$. È il motivo per cui la traccia richiede una griglia bidimensionale, ed è ciò che la campagna C6 misura direttamente confrontando tutte le fattorizzazioni di P , forme degeneri comprese.
- **Perché la forma ottima dipende da $M:N$.** Per $M = N$ la griglia migliore è quella quadrata, cioè proprio quella che `grid_default_shape` sceglie da sé; per $M = 3N$ conviene invece $P_r > P_c$. La campagna C1 varia il rapporto $M:N$ a lavoro costante e rende visibile questo effetto separandolo da quello della taglia.

Un'osservazione che va tenuta presente leggendo il weak scaling (campagna C5): su una griglia 2D con blocco locale costante, M segue P_r e N segue P_c , quindi cambiare P cambia *anche* il rapporto $M:N$. È inerente alla decomposizione, ed è la ragione per cui l'effetto della forma viene misurato separatamente in C1 e C6.

Capitolo 5

Il nucleo di CPU

Il backend `scheme_a` è l'implementazione di riferimento del progetto: è il denominatore di ogni speedup, ed è l'unica che gira ovunque, anche su una macchina senza `nvcc`. È scritta in C11 portabile e non usa intrinsics: l'unica leva sulla vettorizzazione è la forma del codice, che deve essere abbastanza semplice da lasciare al compilatore la decisione.

5.1 Specializzazione su k

Nello schema A i k accumulatori sono variabili locali del ciclo su j . Perché il compilatore possa tenerli in registro deve conoscerne il numero a tempo di compilazione: con k noto solo a runtime, `acc` diventa per forza un array in memoria e ogni aggiornamento un accesso.

Il codice genera quindi cinque funzioni specializzate — una per ciascuno dei k richiesti dalla traccia — da una singola definizione via macro:

```
#define COLS_8(M)  COLS_6(M) M(6) M(7)
#define DECLARE_ACC(c) scalar_t acc##c = (scalar_t)0;
#define UPDATE_ACC(c)  acc##c += a * xrow[c];
#define STORE_ACC(c)   yrow[c] = acc##c;

/* per ogni riga i: */
COLS(DECLARE_ACC)
for (j = 0; j < n_loc; j++) {
    const scalar_t a = arow[j];          /* letto UNA volta */
    const scalar_t *restrict xrow = X_loc + (size_t)j * ldx;
    COLS(UPDATE_ACC)                    /* k aggiornamenti espliciti */
}
COLS(STORE_ACC)
```

Listato 5.1: Generazione delle specializzazioni (estratto di `src/kernel/scheme_a.c`).

Il percorso caldo risultante contiene istruzioni C esplicite per ogni colonna, senza alcun ciclo interno con estremo runtime: gli accumulatori sono variabili scalari distinte, non un array indicizzato.

Per ogni altro k interviene il **fallback generico**, che blocca le colonne a gruppi di $KB = 32$:

$$\text{passaggi su } A = \left\lceil \frac{k}{KB} \right\rceil. \quad (5.1)$$

Per $k \leq 32$ il ciclo esterno sui blocchi è degenero e A viene letta una volta sola, quindi il fallback si differenzia dalle specializzazioni soltanto per l'array di accumulatori; per $k = 40$

e $k = 64$ costa invece due passaggi completi su A e su X . La distinzione è importante nel leggere i risultati di sezione 10.2: le righe $k = 40$ e $k = 64$ non sono confrontabili con quelle $k \leq 32$ senza tenerne conto.

5.1.1 Il microbenchmark che misura la specializzazione

Compilando con `FORCE_GENERIC_K=1` le cinque specializzazioni non vengono nemmeno generate e ogni k passa dal fallback. I due binari coesistono (`??`) e `kernel_name()` li distingue nel CSV (`scheme_a` contro `scheme_a_generic`), quindi il confronto è una sola campagna, sullo stesso problema e nella stessa sessione di misura.

La campagna include un **controllo di consistenza**: per $k = 40$ e $k = 64$ entrambe le build passano dal fallback, quindi le due righe devono coincidere entro il rumore. Se non coincidono, la differenza misurata sugli altri k non è specializzazione ma rumore, e la campagna va rifatta.

5.2 Perché $KB = 32$

Il valore copre tutti i k del collaudo, ma non è questa la ragione sufficiente. Con KB accumulatori in doppia precisione, il compilatore deve tenere KB valori vivi attraverso l'intero ciclo su j : su un'architettura x86-64 con AVX2 sono $KB/4$ registri vettoriali su 16 disponibili, e su AVX-512 $KB/8$ su 32. Al crescere di KB il riuso di A cresce (meno passaggi) ma cresce anche la pressione sui registri, fino allo spill. Il valore 32 è il compromesso che tiene i cinque k richiesti in un passaggio solo; la sua adeguatezza sui k grandi è proprio uno degli oggetti della campagna C3, ed è discussa in sezione 10.2.

5.3 Ciò che il compilatore vede

La build usa `-O3` con `-march=native` (o `-mcpu=native` su aarch64: il Makefile prova quale delle due il compilatore accetta), più `-Wall -Wextra -Wpedantic`. Nessun `#pragma`, nessun `__intrinsic`.

Tre proprietà del codice rendono la vettorizzazione possibile senza aiuti espliciti:

- **Nessun aliasing.** A , X e Y sono dichiarati `restrict`. La garanzia viene ripristinata dentro `local_gemm` con copie locali dei puntatori: il contesto opaco tiene il puntatore ad A come membro, e senza questa copia i kernel specializzati perderebbero l'informazione che avevano quando A era un parametro.
- **Nessuna dipendenza portata dal ciclo interno.** I k aggiornamenti `acc[c] += a * X[j][c]` sono indipendenti fra loro: il ciclo su c è vettorizzabile e, essendo srotolato dalle macro, è già in forma di catene di FMA indipendenti.
- **Accessi a passo unitario.** Come osservato in sezione 2.2, sia la riga di A sia la riga di X sono contigue.

Da completare con i dati della campagna

Verificare sul server l'effettiva vettorizzazione del percorso caldo, per esempio con `gcc -fopt-info-vec` oppure ispezionando l'assembly di `kernel_k8` e `kernel_k32`. Il punto da documentare è se e a quale k compaiono spill degli accumulatori: è una delle due spiegazioni candidate del comportamento misurato in sezione 10.2.

5.4 Il costo nascosto: la riletture di X

Lo schema A legge A una volta sola, ed è il suo pregio. Ma per ogni riga i di A percorre **tutta** X : il traffico in lettura su X vale

$$\text{elementi di } X \text{ letti} = m_{\text{loc}} \cdot n_{\text{loc}} \cdot k, \quad (5.2)$$

cioè k volte il traffico su A . Finché X resta nella cache di livello basso questo traffico non arriva in memoria centrale e non si vede; quando X non ci sta più, il costo per elemento di A smette di essere governato dal calcolo e comincia a essere governato dalla banda del livello di cache in cui X è finita.

La soglia è esplicita: X_{loc} occupa $n_{\text{loc}} \cdot k \cdot s$ byte. Con $n_{\text{loc}} = 10\,000$ e $s = 8$ si passa da 640 kB per $k = 8$ a 1,6 MB per $k = 20$ e 2,56 MB per $k = 32$: nella stessa finestra di k in cui il progetto lavora, X attraversa il confine di una cache privata tipica. È la chiave di lettura dei risultati di sezione 10.2, ed è anche la direzione naturale per un'ottimizzazione futura (blocking del ciclo su j , sezione 11.3), che il progetto ha identificato ma non implementato.

Capitolo 6

I backend CUDA

Il progetto contiene quattro kernel CUDA e un riferimento esterno. Non sono alternative in competizione: sono una **sequenza**, in cui ogni versione cambia una sola cosa rispetto alla precedente, così che ogni differenza di tempo misurata abbia una causa identificabile.

Tabella 6.1: I backend GPU e la variabile che ciascuno cambia.

Backend	Mappatura	Che cosa cambia rispetto al precedente
cuda_naive	un thread per elemento di Y	baseline: riuso di A in registro perso
cuda_warp	un warp per riga di Y	recupera riuso e coalescenza; k accumulatori per lane
cuda_warp_smem	come sopra	la fetta di X passa dalla shared memory
cuda_warp_tiled	un warp per tile di colonne di una riga	meno accumulatori per lane, più letture di A
cublas	cublas<t>gemm	riferimento esterno, non una proposta

6.1 Struttura comune

Tutti condividono il ciclo di vita di sezione 3.2.1. La ripartizione fra preprocessing e cammino cronometrato è la stessa per tutti, ed è la condizione perché i loro tempi siano confrontabili:

Fase	Dove	Cronometro
creazione del contesto CUDA	local_gemm_create	no: $t_{\text{setup}}^{\text{init}}$
tutti i cudaMalloc in VRAM	local_gemm_create	no: $t_{\text{setup}}^{\text{alloc}}$
H2D di A , una volta sola	local_gemm_create	no: $t_{\text{setup}}^{\text{H2D } A}$
H2D di X , kernel, D2H di Y	local_gemm	sì: t_{local}
solo il kernel (cudaEvent)	dentro local_gemm	sì: t_{kernel}
solo l'H2D di X (cudaEvent)	dentro local_gemm	a parte: $t_{\text{H2D } X}$
solo la D2H di Y (cudaEvent)	dentro local_gemm	a parte: $t_{\text{D2H } Y}$

Tabella 6.2: Che cosa entra in quale cronometro, uguale per tutti i backend GPU.

Perché i cudaEvent e non l'orologio dell'host. Un lancio di kernel è asincrono: l'orologio dell'host misurerebbe il tempo di *accodamento*, non quello di esecuzione. Solo il backend può misurare t_{kernel} , e solo con event registrati sullo stream. Per questo l'interfaccia espone local_gemm_last_compute_seconds: è il canale con cui un backend che sa distinguere il kernel dal resto pubblica il dato, e i backend di CPU restituiscono un valore negativo per dire “non applicabile”.

Lo stesso vale, e con un argomento ancora più stringente, per la D2H di Y : quella copia parte solo quando il kernel ha finito, quindi un cronometro sull’host misurerebbe “attesa del kernel + copia” e attribuirebbe al PCIe del tempo di calcolo. Gli event non sono qui una raffinatezza, sono la condizione perché il numero significhi ciò che dice.

La fase locale è scomposta per intero. La differenza $t_{\text{local}} - t_{\text{kernel}}$ resta nel CSV come *transfer/runtime overhead*, ma non è più l’unica informazione disponibile, e da sola non era *discutibile*: metteva nello stesso numero il PCIe e il runtime dell’host, che hanno cause diverse e scalano in modo diverso — il primo con $n_{\text{loc}}k$ e $m_{\text{loc}}k$, il secondo per niente. I due trasferimenti hanno quindi un canale ciascuno, e l’overhead del runtime è ciò che **avanza**:

$$t_{\text{local}} = t_{\text{H2D } X} + t_{\text{kernel}} + t_{\text{D2H } Y} + t_{\text{launch}}. \quad (6.1)$$

Solo un backend che misura tutte e tre le voci nominate può produrre t_{launch} ; altrove resta la sentinella negativa. Nessuna delle voci di destra oltre t_{kernel} entra nella metrica ufficiale: sono esattamente i tempi che la traccia esclude da T e consente di misurare e discutere a parte (sezione 8.3).

Dai tempi alla banda. I tre trasferimenti espongono anche i *byte* trasferiti (`local_gemm_bytes_h2d_A` e le due per invocazione), così che il driver possa pubblicare `bw_h2d_A_gbs`, `bw_h2d_X_gbs` e `bw_d2h_Y_gbs` e confrontarle con il picco del bus. Senza i byte, un tempo di copia non si può giudicare: dice che la copia costa, non se stia andando alla velocità giusta.

Anche il preprocessing è scomposto. Il totale t_{setup} dice che la preparazione costa, non *perché*. Su CUDA le tre voci che lo compongono hanno cause e andamenti diversi, e vanno separate per essere discusse:

Voce	Che cos’è e come scala
$t_{\text{setup}}^{\text{init}}$	creazione del contesto CUDA: costo fisso di qualche centinaio di millisecondi, indipendente dalla taglia
$t_{\text{setup}}^{\text{alloc}}$	i <code>cudaMalloc</code> di A , X e Y in VRAM: cresce con la taglia, ma non è un trasferimento
$t_{\text{setup}}^{\text{H2D } A}$	la copia H2D di A : l’unica delle tre che sia PCIe e l’unica che scali con MN

Tabella 6.3: Scomposizione del preprocessing del backend GPU.

L’ultima è il numero più interessante da riportare, perché è esattamente il trasferimento che la scelta di separare `create` da `local_gemm` (sezione 3.2.1) ha tolto dal cammino misurato: quantificarlo è il modo onesto di dire quanto valga quella scelta. Le tre voci sommate sono $\leq$ del totale — restano fuori i controlli sugli argomenti e la creazione degli event — e valgono 0 sui backend di CPU, dove anche il totale è dell’ordine del microsecondo.

Capienza della VRAM. Non viene stimata prima, si verifica tentando. La catena di `cudaMalloc` si ferma alla prima allocazione che non entra, e solo allora `cudaMemGetInfo` viene interrogata per comporre un messaggio che dice quanta memoria serviva e quanta ce n’era. Un controllo preventivo confronterebbe una fotografia già obsoleta — sul server più rank MPI condividono l’unica GPU — e andrebbe corretto con un margine arbitrario. Un blocco locale di $40\,000 \times 40\,000$ in doppia precisione richiede 12,8 GiB contro i 15,5 GiB della scheda, e il programma si ferma prima di allocare.

Un solo device. Il server di dipartimento ha una sola GPU: tutti i rank usano il device 0 e non esiste logica di selezione. La conseguenza sperimentale è importante ed è discussa in sezione 6.7.

6.2 `cuda_naive`: la baseline

Il thread t calcola $Y[i][c]$ con $i = t/k$ e $c = t \bmod k$, scorrendo tutta la dimensione j con un solo accumulatore. Serve a due cose, in quest'ordine: validare la pipeline (build con `nvcc`, ciclo di vita a tre fasi, trasferimenti, misura con gli event) prima che esista il kernel che conta; e dare un numero di partenza. Uno speedup senza baseline non è un risultato.

Le sue due debolezze sono volute, perché sono esattamente ciò che le versioni successive devono battere:

- **il riuso di A in registro è perso:** ogni thread tiene un solo accumulatore, quindi la stessa riga di A viene richiesta k volte, una per ciascuno dei k thread della riga. Le richieste sono simultanee e la cache ne assorbe quasi tutte, ma il lavoro c'è — ed è il ritorno allo schema B della eq. (2.3);
- **la coalescenza è parziale:** thread consecutivi hanno c consecutivo e leggono $X[j][c]$ contigui, cioè 24 B per riga di X con $k = 3$ e 256 B con $k = 32$: da meno di una transazione a due transazioni piene.

6.3 `cuda_warp`: un warp per riga

Il kernel assegna un warp a ogni riga di Y e distribuisce il ciclo su j fra le 32 lane: la lane l visita $j = l, l + 32, l + 64, \dots$. Il ribaltamento è completo rispetto alla baseline:

- lane consecutive leggono elementi consecutivi di A : coalescenza piena;
- ogni lane tiene *tutti* e k gli accumulatori in registro e riusa ogni valore di A caricato per aggiornarli tutti: è lo schema A, trasposto sul warp;
- i risultati parziali delle 32 lane vengono ridotti dentro il warp con `__shfl_down_sync`, senza shared memory e senza barriere; la lane 0 scrive la riga di Y .

I cinque k della traccia sono istanze template distinte, così che il numero di accumulatori sia una costante di compilazione — lo stesso motivo per cui il nucleo di CPU genera cinque funzioni. Ogni altro k attraversa un fallback runtime a tile di quattro colonne, senza limite massimo su k .

Una nota di correttezza che vale la pena isolare: la riduzione usa la maschera piena e non `__activemask()`. Da Volta in poi le lane di un warp possono divergere e riconvergere in modo indipendente, e la documentazione CUDA dice esplicitamente che il valore di `__activemask()` non implica quali lane parteciperanno alla shuffle. Per questo il kernel è scritto in modo che tutte e 32 le lane di un warp valido raggiungano le shuffle, anche nell'ultimo blocco o quando n_{loc} è minore di 32: i warp interamente fuori intervallo escono prima, quelli parziali no.

6.3.1 Un secondo layout per X : `X_LAYOUT=column`

In `cuda_warp` la lane l visita $j = l, l + 32, \dots$, quindi le letture della riga di A sono perfettamente coalescenti — lane consecutive, indirizzi consecutivi. Le letture di X , invece, non lo sono: con X row-major la lane l legge $X[j][c]$ e due lane consecutive distano `ldx` scalari. Il traffico c'è ma è sparso, e il numero di transazioni per warp cresce.

Il rimedio non richiede di toccare il kernel: basta cambiare l'*ordine in memoria* del blocco locale di X . Con `X_LAYOUT=column` il driver, in preprocessing, riscrive X_{loc} per colonne, così

che

$$X[j][c] = \mathbf{x_loc}[c \cdot n_{\text{loc}} + j], \quad (6.2)$$

e allo stesso passo c le 32 lane leggono n_{loc} elementi consecutivi: anche X diventa coalescente. La selezione è una costante di compilazione, quindi non c'è nessun ramo nel percorso caldo, e la conversione è già terminata prima che il backend venga chiamato.

Tre conseguenze da enunciare, perché sono il motivo per cui l'opzione è limitata e non generale:

- **vale solo per `cuda_warp`**. Il `Makefile` la accetta solo lì e gli altri backend rifiutano `column`: attivarla altrove darebbe risultati sbagliati in silenzio, che è proprio la categoria di errore che il progetto si impegna a rendere impossibile;
- **X deve essere compatta ($\text{ldx} = k$)**: in questa build `ldx` descrive la capienza $n_{\text{loc}} \cdot \text{ldx}$ del buffer, non lo stride fra colonne, che vale n_{loc} . Il padding di X non è ammesso, e il backend lo verifica;
- **la conversione ha un costo**, ed è cronometrata a parte (`t_prep_convert_X_s`). È preprocessing — avviene una volta sola, mentre X viene poi riusata per tutte le repetition — ma riportarla è ciò che rende il confronto fra i due layout una misura completa e non solo un confronto fra due tempi di kernel.

Le due build coesistono come binari distinti (`-xcol`) e `kernel_name()` riporta il suffisso (`xcol`), quindi le righe restano distinguibili nel CSV, che porta comunque anche una colonna `x_layout` esplicita.

6.4 `cuda_warp_smem`: la fetta di X in shared memory

È `cuda_warp` con **una sola variabile cambiata**: da dove arrivano gli elementi di X . Ordine dei cicli, lettura singola di A , accumulatori in registro, riduzione per shuffle e specializzazione sui cinque k restano identici. È questo che rende il confronto una misura.

Il difetto attaccato. In `cuda_warp` ogni warp percorre tutta X , e i warp sono m_{loc} : il traffico in lettura su X vale $m_{\text{loc}} \cdot n_{\text{loc}} \cdot k$ elementi contro i $m_{\text{loc}} \cdot n_{\text{loc}}$ di A . X è piccola e sta in L2, quindi il traffico non arriva in DRAM; ma è banda L2, e cresce linearmente con k . È la trasposizione esatta, sulla gerarchia della GPU, dello stesso fenomeno descritto per la CPU in sezione 5.4.

Il rimedio. Gli 8 warp di uno stesso blocco lavorano su righe *diverse* di A , ma allo stesso passo j vogliono le *stesse* righe di X . Il blocco stagia quindi cooperativamente un tile $\mathbf{x_rows_per_tile} \times k$ di X in shared memory e i suoi warp lo riusano: il traffico verso L2 si divide per il numero di warp per blocco.

$$\text{traffico L2 su } X : \underbrace{m_{\text{loc}} n_{\text{loc}} k}_{\text{cuda_warp}} \longrightarrow \underbrace{\frac{m_{\text{loc}} n_{\text{loc}} k}{W}}_{\text{cuda_warp_smem}}, \quad W = \text{warp per blocco.} \quad (6.3)$$

Ipotesi: il guadagno può essere nullo, e sarebbe comunque un risultato

Se in FP64 il kernel è già compute-bound — come il roofline di sezione 7.1 suggerisce per $k \gtrsim 4$ — ridurre il traffico L2 non può dare alcun guadagno. La misura serve a dimostrarlo, non a nascondere: un guadagno nullo qui è una conferma del modello, e va riportato come tale. La stessa misura in singola precisione, dove il modello prevede un regime memory-bound, è il controllo incrociato naturale.

6.4.1 Il conflitto sui banchi e il padding a $k + 1$

La shared memory ha 32 banchi da 32 bit. Un `double` ne occupa **due**. La lane l legge `X_tile[l*tile_row_stride + c]`, quindi fra due lane consecutive ci sono `tile_row_stride` scalari, cioè $2 \cdot \text{tile_row_stride}$ banchi:

<code>tile_row_stride</code>	banchi di stride	effetto
$k = 32$	$64 \equiv 0 \pmod{32}$	tutte e 32 le lane sullo stesso banco: conflitto a 32 vie
$k + 1 = 33$	$66 \equiv 2 \pmod{32}$	stride dispari in <code>double</code> : conflitto minimo

È l'analogo esatto del padding $\text{LDA} = N + 24$ studiato sulla CPU contro i conflict miss di L1: stesso fenomeno, altra gerarchia di memoria. Il padding è un parametro di compilazione (`SMEM_PAD`) proprio perché i due casi vanno *misurati* entrambi, e i due binari coesistono (??): non c'è modo di misurare per sbaglio la build sbagliata.

6.4.2 Che cosa il tile cambia davvero, e che cosa no

Il tile pesa $\text{x_rows_per_tile} \cdot (k + \text{SMEM_PAD}) \cdot s$ byte e k è noto solo a runtime: la shared memory è quindi dinamica e il numero di righe si sceglie in `local_gemm_create`, una volta sola, fuori dal cammino cronometrato.

Il punto da chiarire prima di tutto è quale grandezza il tile governi, perché l'intuizione naturale è sbagliata:

Il tile non cambia il traffico verso L2

Il fattore di riuso è il numero di warp per blocco, e basta. Ogni blocco legge comunque tutta la sua fetta di X una volta, che lo faccia in due tile grandi o in venti piccoli: il traffico resta $m_{\text{loc}} n_{\text{loc}} k / W$ in ogni caso.

Ciò su cui il tile agisce davvero è il **numero di barriere** e l'ILP. Ogni tile costa due `__syncthreads()`, e un tile che contiene un solo passo di warp (`x_rows_per_tile = 32`) fa arrivare ogni warp alla barriera successiva dopo una sola iterazione del ciclo su j , senza nessuna iterazione con cui coprire la latenza delle letture dalla shared memory. Dimezzare il numero di tile dimezza le barriere; non tocca il traffico.

Il budget non si sceglie in assoluto. La domanda giusta non è “quanti KiB dare a un blocco” ma “quanti gliene servono perché la shared *non sia lei* il vincolo attivo sull'occupancy”. La differenza è sostanziale: a $k = 32$ i 32 accumulatori costano già 64 registri a 32 bit per thread, quindi il vincolo attivo sono quasi certamente i registri — e un budget di shared scelto a mano più stretto starebbe rimpicciolendo il tile per proteggere un'occupancy già persa altrove.

La funzione `plan_tile` risolve il problema dell'uovo e della gallina — l'API dell'occupancy vuole i byte di shared che stiamo calcolando — interrogandola con `smem = 0`:

1. `cudaOccupancyMaxActiveBlocksPerMultiprocessor` con shared memory “gratis” dà quanti blocchi consentono *registri e thread per SM da soli*: è l'obiettivo da non peggiorare;
2. il budget per blocco è la shared memory per SM diviso quell'obiettivo (oppure il valore forzato da `SMEM_BUDGET_BYTES`), limitato dal tetto architetturale di 48 KiB oltre il quale il *lancio* fallirebbe;
3. le righe che ci stanno vengono arrotondate per difetto a `TILE_GRANULARITY`, con un minimo di un passo di warp completo e senza mai allocare più righe di quante X ne abbia davvero;

4. il piano viene **verificato invece che creduto**: il driver riserva shared memory per blocco oltre a quella richiesta, quindi il passo 2 può essere ottimista. Si reinterroga l'API con i byte effettivi e si scende di un passo finché l'obiettivo è raggiunto. La riserva del driver non viene modellata con una costante inventata: viene scoperta.

Il piano risultante finisce nel CSV come `x_rows_per_tile` e `blocks_per_sm`. Non sono metriche di prestazione: sono la configurazione effettivamente scelta, e senza di esse una campagna sul tiling mostrerebbe curve che salgono e scendono senza poter distinguere un cambio di tile da un cambio di occupancy.

La granularità è un compromesso, non un requisito. `TILE_GRANULARITY` vale 32 per default, ma non è una condizione di correttezza: il kernel gestisce già i tile parziali, sia nel caricamento cooperativo sia nel ciclo di consumo. È soltanto l'ottimizzazione “nessuna lane inattiva nell'ultimo passo di warp”, e ha un costo misurabile: a $k = 32$ in doppia precisione le righe che entrano nel budget sono 62, e arrotondare a 32 ne butta via metà, raddoppiando le barriere per guadagnare zero lane. Da qui il parametro: costo e beneficio si misurano, non si assumono.

Per lo stesso motivo non esiste più un massimo espresso in *righe*: non era un vincolo fisico e, appena il budget cresce, sarebbe diventato silenziosamente il vincolo attivo ai k piccoli, rendendo inefficace qualunque sweep sul budget. Il solo tetto rimasto è quello architetturale in byte.

6.4.3 Perché non ci sono uscite anticipate

`cuda_warp` può fare `if (warp_id >= m) return;` in cima. Qui non si può: il ciclo sui tile contiene `__syncthreads()`, che è una barriera di *blocco*. Se i warp dell'ultimo blocco — quello parzialmente pieno — uscissero prima, i rimanenti aspetterebbero a una barriera che nessuno raggiunge più. Tutti i thread del blocco percorrono quindi lo stesso numero di iterazioni e di barriere; sono soltanto l'accumulo e la scrittura finale a essere condizionati da un flag `active`.

6.5 `cuda_warp_tiled`: meno accumulatori per lane

L'ultima variante sposta il compromesso nella direzione opposta: un warp calcola un tile di `WARP_COL_TILE` colonne di una sola riga, invece di tutte e k . Le lane continuano a visitare $j = l, l + 32, \dots$ come prima.

Detto $T = \text{WARP_COL_TILE}$:

$$\text{warp per riga} = \left\lceil \frac{k}{T} \right\rceil, \quad \text{accumulatori per lane} = T, \quad \text{letture di } A \times \left\lceil \frac{k}{T} \right\rceil. \quad (6.4)$$

Il tile è un parametro di compilazione (`WARP_COL_TILE`, default 8) e compare nel nome del kernel, quindi lo sweep sui valori 4, 8, 16, 32 produce righe distinguibili nello stesso CSV. È la variante che serve a rispondere a una domanda precisa: per $k = 32$, tenere 32 accumulatori per lane costa in occupancy più di quanto costi rileggere A quattro volte? Il modello non risponde da solo, perché i due effetti hanno segno opposto; la campagna C2 lo misura.

6.6 `cublas`: il riferimento esterno

Non è una proposta, è la risposta a “perché avete scritto un kernel invece di chiamare una libreria?”. La risposta ha valore solo se il numero della libreria è stato misurato sulla stessa macchina, con la stessa pipeline, gli stessi trasferimenti e lo stesso cronometro: per questo

cuBLAS entra come un backend qualsiasi, dietro la stessa interfaccia, e non come uno script a parte.

6.6.1 Riga contro colonna: nessuna trasposizione

cuBLAS è column-major, il progetto è row-major. La soluzione non è trasporre — costerebbe più del prodotto — ma reinterpretare: una matrice $P \times Q$ row-major con leading dimension ld è *già*, bit per bit, la matrice $Q \times P$ column-major con la stessa ld . Leggere gli stessi byte con l'altra convenzione equivale a trasporre gratis. Quindi

$$Y_{cm} = (Y_{rm})^T = (A_{rm} X_{rm})^T = (X_{rm})^T (A_{rm})^T = X_{cm} A_{cm}, \quad (6.5)$$

e nella convenzione $C(M \times N) = A(M \times K) B(K \times N)$ la chiamata diventa $M = k$, $N = m_{loc}$, $K = n_{loc}$, con dX e dA in quest'ordine ed entrambe le operazioni a CUBLAS_OP_N. I vincoli di cuBLAS sulle leading dimension si traducono in $ldx \geq k$, $lda \geq n_{loc}$, $ldy \geq k$: esattamente le tre condizioni che `local_gemm_create` già verifica.

L'handle cuBLAS è creato in `local_gemm_create`: la sua creazione è costosa perché carica i kernel della libreria, e metterla nel cammino misurato misurerebbe l'inizializzazione della libreria invece del suo GEMM.

6.6.2 L'attesa è che vada male

cuBLAS è un GEMM general-purpose, ottimizzato per matrici in cui tutte e tre le dimensioni sono grandi. Qui $k \leq 32$: la libreria non ammortizza il proprio percorso di selezione dell'algoritmo, e il tiling che sceglie ha molte più righe che colonne utili. Un numero basso in questa riga è l'argomento più forte a sostegno dell'intero progetto — purché sia misurato, ed è ciò che C2 fa.

Da completare con i dati della campagna

Profilare con `nsys` o `ncu` quale kernel interno cuBLAS seleziona per $k \in \{3, 8, 32\}$, e verificare empiricamente la geometria del tile scelto. È il dato che trasforma “va male” in “va male *perché*”.

6.7 Più rank MPI sulla stessa GPU

Con più rank MPI sullo stesso nodo i contesti CUDA si alternano sulla scheda, perché MPS non è attivo. Il risultato resta corretto, ma t_{kernel} comprende anche il tempo in cui il contesto di un rank è sospeso a favore di un altro.

La conseguenza sulla lettura dei numeri è netta, e va enunciata prima di presentarli: **con $P > 1$ su una sola GPU, t_{kernel} non è il tempo di una GPU dedicata, ma il throughput aggregato della scheda sul problema globale.** Per caratterizzare un kernel in sé — confronto fra backend, sweep su BLOCK, effetto del padding in shared memory — va usato `-np 1`, che sulla stessa scheda esegue lo stesso lavoro totale senza contesa fra contesti. Tutte le campagne di caratterizzazione del kernel (C2, C3, C1 nella variante GPU) sono infatti a rank singolo.

Da completare con i dati della campagna

Verificare sul nodo di calcolo (non sul frontend) il supporto CUDA-aware di Open MPI con `ompi_info | grep cuda` e il numero di GPU con `nvidia-smi`. Se il supporto è presente, l'interfaccia `local_gemm` non richiede riscritture per passare puntatori device alle collettive: è stata progettata agnostica rispetto alla posizione della memoria proprio per questo, ed è un lavoro futuro (sezione 11.3).

Capitolo 7

Modello roofline

Il modello roofline mette in relazione l'intensità aritmetica di un nucleo con le due sole cose che possono limitarlo, la potenza di calcolo e la banda di memoria:

$$P_{\text{ottenibile}}(I) = \min(P_{\text{picco}}, B \cdot I), \quad I_{\text{crossover}} = \frac{P_{\text{picco}}}{B}. \quad (7.1)$$

Serve qui a due scopi: dire *prima* della misura quale regime aspettarsi per ogni k , e dare un metro con cui giudicare i numeri ottenuti. Un kernel che raggiunge il 70% del tetto pertinente è un buon kernel; lo stesso numero assoluto, confrontato con il tetto sbagliato, non dice nulla.

Dalla eq. (2.3), con lo schema A e contando il solo traffico su A :

$$I(k) = \frac{2k}{s} = \begin{cases} k/4 & \text{FP64 } (s = 8) \\ k/2 & \text{FP32 } (s = 4) \end{cases} \quad (7.2)$$

Tabella 7.1: Intensità aritmetica del nucleo, in FLOP/B, per i k della traccia.

	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
FP64	0,75	1,50	2,00	5,00	8,00
FP32	1,50	3,00	4,00	10,00	16,00

7.1 Roofline della GPU

Da completare con i dati della campagna

I valori di picco in tabella 7.2 sono quelli nominali della Quadro RTX 5000 (TU104). Confermarli sul nodo di calcolo con `deviceQuery` (clock, numero di SM, larghezza e frequenza del bus di memoria) e, per la banda effettivamente raggiungibile, con una misura tipo STREAM su device: il rapporto fra banda nominale e banda misurata è tipicamente 0,75–0,85, e usare quella misurata sposta il punto di crossover.

Tabella 7.2: Picchi nominali della GPU di misura e punti di crossover derivati.

Grandezza	Valore	Origine
CUDA core	3072	specifica TU104
Clock boost	1,815 GHz	specifica
Picco FP32	11,1 TFLOP/s	$2 \times 3072 \times f$
Picco FP64	0,35 TFLOP/s	FP32/32 (rapporto Turing)
Banda di memoria	448 GB/s	GDDR6, 256 bit, 14 Gbit/s
Memoria	15,5 GiB	disponibile
Crossover FP64	0,78 FLOP/B	P_{picco}/B
Crossover FP32	24,9 FLOP/B	P_{picco}/B

Il confronto fra tabella 7.1 e tabella 7.2 dà la previsione, ed è il risultato più utile di tutto il modello:

Previsione

- **In FP64** il crossover cade a $I = 0,78$, cioè a $k \approx 3,1$. Il nucleo è quindi *memory-bound solo per $k = 3$* — e appena — e **compute-bound per ogni $k \geq 6$** . Il tetto pertinente per i quattro k più grandi è 350 GFLOP/s, e non dipende da k : le curve GFLOPS(k) dei backend GPU in doppia precisione dovrebbero appiattirsi.
- **In FP32** il crossover cade a $I = 24,9$, cioè a $k \approx 50$. Il nucleo è quindi **memory-bound per tutti i k della traccia**, e il tetto vale $B \cdot k/2 = 224k$ GFLOP/s: cresce *linearmente* con k e per $k = 32$ arriva a 7,2 TFLOP/s, venti volte il tetto FP64.

Le due precisioni si trovano quindi in regimi *qualitativamente* diversi, e questo è l'argomento quantitativo con cui il progetto discute la scelta della precisione: la doppia non costa il fattore 2 del traffico, costa il fattore 32 del picco FP64 di una scheda Turing. La previsione ha una conseguenza direttamente misurabile, ed è quella che la campagna C7 (sezione 8.5.7) mette alla prova: il rapporto fra i tempi nelle due precisioni deve **crescere con k** . Se fosse piatto e vicino a 2, sarebbe questo modello a essere sbagliato.

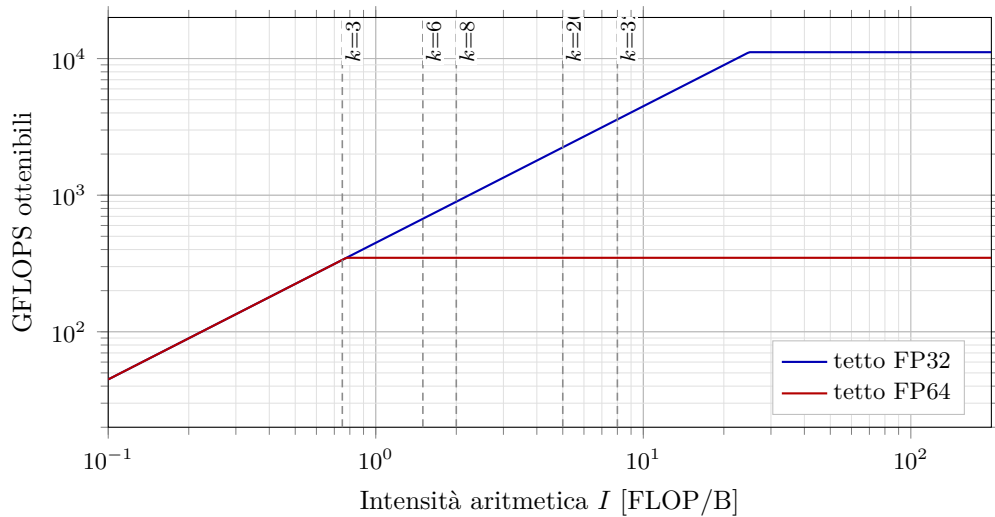


Figura 7.1: Roofline della GPU di misura. Le verticali tratteggiate sono le intensità aritmetiche FP64 dei cinque k della traccia. In FP64 solo $k = 3$ cade a sinistra del ginocchio; in FP32 tutti i k della traccia restano nella regione limitata dalla banda.

7.1.1 Che cosa il modello non dice

Due limiti vanno enunciati, perché altrimenti il modello verrebbe usato per concludere più di quanto possa.

Il primo: l'intensità della eq. (7.2) conta il solo traffico su A verso la DRAM. Il traffico su X , che vale $m_{\text{loc}} n_{\text{loc}} k$ elementi (eq. (5.2)), non compare perché X sta in L2 — ma esiste, e consuma banda L2. Il roofline classico a un solo livello non lo vede; è esattamente il traffico che `cuda_warp_smem` riduce (sezione 6.4), e la ragione per cui un kernel può restare sotto il tetto pur avendo l'intensità giusta.

Il secondo: il tetto di calcolo presuppone che ci sia abbastanza parallelismo per saturare gli SM. Con un warp per riga di Y , i warp disponibili sono m_{loc} : una matrice larga e bassa (rapporto $M:N$ sfavorevole) può affamare la GPU di parallelismo pur avendo la stessa intensità aritmetica. È precisamente il motivo per cui la campagna C1 varia il rapporto $M:N$ a *lavoro costante*: separa l'effetto della forma da quello della taglia.

7.2 Roofline della CPU

Da completare con i dati della campagna

Completare tabella 7.3 con i dati del server: modello del processore, numero di core fisici per socket, frequenza sostenuta con vettorizzazione attiva, larghezza SIMD, dimensione di L1/L2/L3, banda misurata con STREAM a un core e a tutti i core. Il modello del nucleo di CPU non è completabile senza la dimensione di L2, perché la soglia discussa in sezione 5.4 è proprio quella.

Tabella 7.3: Parametri del nodo di calcolo (da completare).

Grandezza	Valore	Come ottenerlo
Modello CPU	—	<code>lscpu</code>
Socket $\times$ core fisici	$2 \times$ —	<code>lscpu</code>
Frequenza sostenuta	—	<code>turbostat</code> o <code>perf</code>
Larghezza SIMD	—(AVX2 / AVX-512)	<code>lscpu grep Flags</code>
L1d / L2 per core, L3 per socket	—	<code>lscpu -C</code>
Banda STREAM, 1 core	—	STREAM triad
Banda STREAM, tutti i core	—	STREAM triad
Picco FP64, 1 core	—	$2 \times \text{SIMD} \times \text{FMA} \times f$

Anche senza questi valori, i dati già raccolti in campagna C3 permettono di ricavare per via inversa le grandezze rilevanti, ed è ciò che sezione 10.2 fa: dal tempo misurato si ottiene la banda effettiva su A e quella su X , e il confronto fra le due individua il livello di gerarchia che sta limitando il nucleo per ciascun k .

Capitolo 8

Metodologia sperimentale

8.1 Ambiente di misura

Tabella 8.1: Ambiente software del server di dipartimento, come registrato nei metadati della campagna C3.

Componente	Versione
Compilatore CUDA	nvcc 12.8 (V12.8.61)
MPI	Open MPI 5.0.9
Architettura GPU di compilazione	sm_75 (Turing)
Compilatore host	mpicc, C11, -O3 -march=native
Compilatore device	nvcc, C++14, -O3 -lineinfo
Modello CPU, cache, frequenza	– [DA FARE: completare con 1scpu]

Il server è un nodo a doppio socket con GPU singola. Il frontend non ha accesso diretto alla GPU: tutte le verifiche d’ambiente (`nvcc -version`, `mpi_info | grep cuda`, `deviceQuery`, `nvidia-smi`) e tutti i benchmark vanno eseguiti sui nodi di calcolo attraverso lo scheduler, non sul nodo di login. È un dettaglio operativo, ma è la causa più frequente di misure invalide.

Binding obbligatorio. Tutte le esecuzioni multi-rank usano `-bind-to core -map-by core`. Senza binding i processi migrano fra i socket, la località NUMA salta e le misure diventano rumore: non è un affinamento, è una precondizione.

8.2 Che cosa entra nel cronometro

Per ogni repetition e per ogni rank vengono misurati i tempi delle tre fasi, poi ridotti fra i rank con `MPI_MAX` — repetition per repetition, prima di qualunque media. Prendere il massimo è la scelta corretta perché le collettive sincronizzano: il tempo del prodotto distribuito è quello del rank più lento, e mediare prima di massimizzare lo sottostimerebbe.

Il tempo ufficiale vale

$$t_{\text{official}} = \begin{cases} t_{\text{total}} & \text{backend CPU} \\ t_{\text{bcast}} + t_{\text{kernel}} + t_{\text{reduce}} & \text{backend CUDA} \end{cases} \quad (8.1)$$

dove t_{total} è misurato direttamente dall’inizio del broadcast alla fine della reduce. Sui backend CUDA la sostituzione di t_{local} con t_{kernel} realizza *esattamente* la prescrizione della traccia: il

tempo di trasferimento da e verso la scheda non entra in T , ma resta misurato e discusso a parte.

Le tre metriche riportate sono:

$$\text{gflops} = \frac{2MNk}{\max_r t_{\text{official}}} \cdot 10^{-9} \quad \text{metrica ufficiale} \quad (8.2)$$

$$\text{gflops_compute} = \frac{2MNk}{\max_r t_{\text{compute}}} \cdot 10^{-9} \quad \text{solo calcolo} \quad (8.3)$$

$$\text{gflops_kernel} = \frac{2MNk}{\max_r t_{\text{kernel}}} \cdot 10^{-9} \quad \text{alias CUDA} \quad (8.4)$$

con $t_{\text{compute}} = t_{\text{local}}$ su CPU e t_{kernel} su CUDA. Sui backend di CPU, dove il kernel è l'invocazione e non c'è nulla da separare, t_{kernel} e **gflops_kernel** valgono -1 : il valore negativo dice “non applicabile” e rende riconoscibili a colpo d'occhio le righe GPU nel CSV. Su GPU la colonna da leggere è **gflops_kernel**; su CPU è **gflops**.

t_{setup} riporta separatamente il preprocessing del backend, che per costruzione sta fuori da tutte e tre le metriche: è riportato, non nascosto.

8.3 I tempi esclusi dalla misura ufficiale

La traccia esclude da T il preprocessing, l'I/O e i trasferimenti da e verso la scheda, ma consente esplicitamente di misurarli e discuterli a parte. Il progetto prende sul serio la seconda metà della frase: il driver raccoglie *tutte* queste voci, in coda al CSV e sotto un titolo proprio nel report leggibile, senza che nessuna di esse entri in t_{official} .

La ragione non è la completezza fine a sé stessa. Con $k \leq 32$ la domanda “quanto costa portare i dati sulla scheda rispetto a calcolarci sopra” non è oziosa: il calcolo per elemento trasferito è piccolo, ed è proprio il tipo di problema in cui il bus può dominare. Un numero che risponda a quella domanda serve, e serve separato.

Tabella 8.2: Le voci escluse dalla metrica ufficiale e raccolte a parte.

Colonna	Che cosa misura
<code>t_prep_alloc_buffers_s</code>	allocazione dei blocchi locali A , X , Y
<code>t_prep_gen_A_s</code>	generazione dei valori di A
<code>t_prep_distrib_A_s</code>	distribuzione MPI di A globale; 0 con <code>-a-mode local</code>
<code>t_prep_gen_X_s</code>	generazione di X
<code>t_prep_distrib_X_s</code>	MPI_Scatterv di X globale; 0 con <code>-x-mode local</code>
<code>t_prep_convert_X_s</code>	conversione di layout di X (sezione 6.3.1)
<code>t_prep_total_s</code>	somma <i>locale</i> delle voci sopra, poi massimo fra i rank
<code>t_setup_s</code>	preprocessing del backend (<code>local_gemm_create</code>)
<code>t_setup_device_init_s</code>	dentro il precedente: creazione del contesto CUDA
<code>t_setup_device_alloc_s</code>	dentro il precedente: i <code>cudaMalloc</code> in VRAM
<code>t_setup_h2d_A_s</code>	dentro il precedente: la sola copia H2D di A
<code>t_h2d_X_mean_s, _std_s</code>	H2D di X per invocazione
<code>t_d2h_Y_mean_s, _std_s</code>	D2H di Y per invocazione
<code>t_launch_overhead_mean_s, _std_s</code>	ciò che resta di t_{local} : runtime dell'host, non PCIe
<code>bw_h2d_A_gbs, bw_h2d_X_gbs,</code> <code>bw_d2h_Y_gbs</code>	banda effettiva dei tre trasferimenti
<code>t_io_csv_raw_s</code>	l'unico I/O del driver: scrittura del CSV per ripetizione

Tre scelte di metodo meritano di essere enunciate, perché cambiano il significato dei numeri:

- **le bande sono aggregate.** Sono byte *sommati su tutti i rank* diviso il tempo del rank più lento. Sul server i rank condividono l'unica scheda, quindi ciò che si vuole sapere è quanto ha trasportato il bus, non quanto ne ha usato un singolo processo;
- **$t_{\text{prep}}^{\text{tot}}$ si compone prima di ridurre.** Ogni voce viene ridotta con `MPI_MAX` fra i rank (cammino critico), ma il totale viene sommato *localmente* e solo dopo ridotto: la somma dei massimi di fasi diverse non è il preprocessing di nessun processo reale;
- **l'unico I/O misurabile è quello di uscita.** Gli input non si leggono da file, si generano dal seme (sezione 2.4): non esiste un tempo di lettura da riportare, ed è una conseguenza diretta della scelta del generatore.

Sui backend di CPU le voci che riguardano la scheda valgono -1 (tempi) o 0 (byte e scomposizione del setup), così una riga di CPU si distingue a colpo d'occhio da una che i trasferimenti li ha davvero. Il CSV per ripetizione riporta le stesse voci non aggregate (`t_h2d_X_s`, `t_d2h_Y_s`, `t_launch_overhead_s`).

Un controllo che il driver fa da solo. Aggiungere righe con le nuove colonne a un CSV prodotto da una build precedente renderebbe il file ambiguo senza alcun errore. Il driver confronta quindi l'intestazione del file esistente con la propria e si ferma se differiscono: è lo stesso principio degli invarianti di capitolo 4 — un errore che non fa fallire nulla va reso impossibile, non evitato con attenzione.

8.4 Stabilità delle misure

Ogni configurazione esegue `-warmup` invocazioni non cronometrate (default 2) seguite da `-reps` cronometrate (10 o 20 secondo la campagna). Per ogni serie il CSV riporta media, mediana, minimo, deviazione standard e coefficiente di variazione

$$\text{CV}\% = 100 \cdot \frac{\sigma}{\mu}. \quad (8.5)$$

Il CV è la colonna che dice, riga per riga, se la media significa qualcosa. Nella campagna C3 vale fra 0,37% e 1,38% (tabella 10.2), quindi le differenze discusse in sezione 10.2 sono ordini di grandezza sopra la dispersione. È un controllo che va rifatto per ogni campagna, non una proprietà acquisita: una riga con CV alto va indagata prima di essere interpretata.

Oltre al CSV aggregato, ogni esecuzione può produrre un CSV *raw* con una riga per repetition, pensato per lo studio della varianza: ha un'intestazione diversa e va unito separatamente.

8.5 Le campagne

Le campagne sono descritte da file `.conf` raggruppati in *suite*, e lanciate dal runner `run_cuda_experiments.sh` in `script/`. Tutti i `.conf` di una campagna condividono la cartella di output e hanno nomi distinti, quindi i loro CSV non si sovrascrivono.

Tabella 8.3: Le sei campagne, nell'ordine di esecuzione consigliato.

ID	Suite	A che domanda risponde	P	Durata
C3	c3_cpu_baseline	qual è il denominatore di ogni speedup, e quanto vale la specializzazione su k	1	5 min
C2	c2_k_sweep	come si comporta ogni backend al variare di k , incluso il fallback generico	1	10 min
C1	c1_size_ratio_{cpu,gpu}	obbligatoria: dimensioni crescenti su 3 rapporti $M:N$	1	8 min
C6	c6_grid_shape	quale forma di griglia conviene a P fisso, cioè perché 2D	8/16/20	6 min
C4	c4_strong_scaling	come scala a problema fisso	1–20	5 min
C5	c5_weak_scaling	quanto costa la sola comunicazione	1–20	5 min
C7.1	c7_1_precision_k	quanto rende davvero la singola precisione, backend per backend e k per k	1	10 min
C7.2	c7_2_precision_resources	<i>perché:</i> registri, spill e shared memory nelle due precisioni	—	3 min

C3 e C2 vengono per primi perché producono i numeri di riferimento che tutte le altre citano. C3, la riga `scheme_a` di C2, C4, C5 e C6 non richiedono `nvcc`: se la build CUDA non funziona, la parte MPI del progetto resta interamente misurabile.

8.5.1 C1 — dimensioni crescenti per rapporto $M:N$

È la campagna esplicitamente richiesta dalla traccia: misure ripetute per M e N crescenti, su almeno tre rapporti, insieme che deve contenere $M = N$.

Tabella 8.4: I dodici punti di C1: quattro passi (fattore 4 in MN) per tre rapporti, scelti in modo che a parità di passo il lavoro sia lo stesso entro il 5%.

Passo	1:1	3:1 ($M = 3N$)	1:2 ($N = 2M$)	MN
s1	3200×3200	5400×1800	2250×4500	$\sim 1,0 \cdot 10^7$
s2	6400×6400	10800×3600	4500×9000	$\sim 4,0 \cdot 10^7$
s3	12800×12800	21600×7200	9000×18000	$\sim 1,6 \cdot 10^8$
s4	25600×25600	43200×14400	18000×36000	$\sim 6,4 \cdot 10^8$

Il disegno è la parte che conta: poiché il lavoro è uguale a parità di passo, le tre curve si sovrappongono sullo stesso asse e la differenza che si legge è la **forma** del blocco, non la taglia. Conta perché il rapporto $M:N$ decide quanto pesa il broadcast di X ($\sim n_{\text{loc}}k$) contro la reduce di Y ($\sim m_{\text{loc}}k$), e su GPU quanti warp esistono (m_{loc}) contro quanto è lungo il ciclo interno (n_{loc}): una matrice larga e bassa affama la GPU di parallelismo.

Al passo s4 la sola A occupa circa 5 GB in doppia precisione: entra nella VRAM da 15,5 GiB, ma è il primo punto che può fallire. Le suite sono in ordine di taglia crescente e hanno `continue_on_error=1`, quindi un eventuale esaurimento della memoria non invalida i punti già raccolti.

8.5.2 C2 — sweep su k , un backend per riga

$k \in \{3, 6, 8, 20, 32\}$ (i cinque della traccia, tutti specializzati) seguiti da $k \in \{1, 7, 17, 40, 64\}$, che attraversano il fallback. La differenza fra le due famiglie di righe è la misura di quanto vale la specializzazione. Su `scheme_a`, $k = 40$ e $k = 64$ costano anche una riletture di A (eq. (5.1)), e vanno letti tenendone conto.

Sei backend sulla stessa griglia di k : `scheme_a`, `cuda_naive`, `cuda_warp`, `cuda_warp_smem`, `cuda_warp_tiled`, `cublas`.

Le colonne in coda al CSV (tabella 8.2) non entrano in nessuna delle metriche di questa campagna, ma vanno lette insieme a esse: sono il materiale con cui si risponde a “quanto costa portare i dati sulla scheda rispetto a calcolarci sopra”. A k piccolo il confronto fra $t_{H2D} X + t_{D2H} Y$ e t_{kernel} è uno dei risultati più istruttivi dell’intera campagna, e si ottiene senza eseguire nulla di aggiuntivo.

8.5.3 C3 — baseline di CPU e specializzazione contro fallback

Due ruoli in una campagna sola: il *denominatore* di ogni speedup della relazione (il nucleo a 1 rank, senza comunicazione e senza GPU), e il microbenchmark `FORCE_GENERIC_K=0` contro `FORCE_GENERIC_K=1` sullo stesso problema. Le due build sono binari distinti con `kernel_name()` diverso, quindi le righe restano distinguibili nel CSV. Il controllo di consistenza su $k = 40$ e $k = 64$ è descritto in sezione 5.1.

8.5.4 C4 — strong scaling

Problema fisso $20\,000 \times 20\,000$ (A da circa 3,2GB), $P \in \{1, 2, 4, 8, 16, 20\}$ sulla forma più quadrata — la stessa che `grid_default_shape` sceglierebbe da sé. A $P = 20$ il blocco locale è 5000×4000 , abbastanza grande da non stare in cache.

$k \in \{3, 32\}$ perché calcolo e comunicazione crescono *entrambi* con k , ma da basi diverse: con $k = 3$ la scalabilità attesa è quasi ideale, con $k = 32$ il broadcast e la reduce pesano dieci volte tanto ed è lì che ci si attende la rottura. Le colonne da riportare sono `gflops` più la terna t_{bcast} , t_{local} , t_{reduce} come frazione di t_{total} .

8.5.5 C5 — weak scaling

Blocco locale costante a 5000×5000 per processo, cioè $M = 5000P_r$ e $N = 5000P_c$. Il lavoro per rank non cambia da $P = 1$ a $P = 20$, quindi il tempo ideale è piatto e tutto ciò che sale è comunicazione pura. Separa calcolo e comunicazione meglio dello strong scaling, dove i due effetti si sovrappongono. Va letta ricordando l’osservazione di sezione 4.6: su una griglia 2D il weak scaling cambia anche il rapporto $M:N$.

8.5.6 C6 — forma della griglia a P fisso

Esegue *tutte* le fattorizzazioni ordinate di P : per $P = 8$ le forme 1×8 , 2×4 , 4×2 , 8×1 ; per $P = 16$ cinque forme; per $P = 20$ sei. È la campagna che giustifica sperimentalmente la griglia **2D** invece di una 1D, cioè il requisito centrale della traccia: le forme degeneri $1 \times P$ e $P \times 1$ sono le due decomposizioni monodimensionali, quindi il confronto è diretto e sullo stesso grafico.

t_{bcast} e t_{reduce} vanno letti **in coppia**: per la eq. (4.4) devono muoversi in direzioni opposte lungo la sequenza di forme, e l’ottimo sta in mezzo. È una previsione falsificabile, ed è il modo in cui la campagna verifica il modello di costo invece di limitarsi a illustrarlo.

8.5.7 C7 — la precisione, misurata e spiegata

È la campagna che mette alla prova la previsione più forte del modello (sezione 7.1): in FP64 il nucleo dovrebbe essere compute-bound per ogni $k \geq 6$ con un tetto indipendente da k , in FP32 memory-bound su tutti i k della traccia con un tetto proporzionale a k . Se il modello regge, il guadagno della singola precisione deve *crescere* con k ; se il guadagno fosse piatto e vicino a 2, sarebbe il modello a essere sbagliato.

È divisa in due perché risponde a due domande diverse, e mescolarle renderebbe entrambe le risposte deboli.

C7.1 — quanto. Dieci configurazioni, cinque backend GPU (`cuda_naive`, `cuda_warp`, `cuda_warp_smem`, `cuda_warp_tiled`, `cublas`) in coppie `double/float` adiacenti, con $M = N = 10\,000$, $P = 1$, `BLOCK=256` e `WARP_COL_TILE=8`. Ogni coppia ha *tutti* gli altri parametri identici, quindi il rapporto

$$S_{32/64}(\text{kernel}, k) = \frac{t_{\text{kernel}}^{\text{double}}}{t_{\text{kernel}}^{\text{float}}} \quad (8.6)$$

isola la sola precisione. Due regole nel calcolarlo: non mescolare tempi di kernel, di fase locale e totali nello stesso rapporto, e guardare anche la dispersione e i campioni per ripetizione, non solo lo speedup puntuale. Per `cuda_warp_smem` vanno lette anche `x_rows_per_tile` e `blocks_per_sm`: il piano di tiling *cambia* con la precisione, perché cambia sia la dimensione dello scalare sia il numero di registri, ed è uno dei casi in cui quelle due colonne fanno la differenza fra un grafico spiegato e uno solo descritto.

C7.2 — perché. Nessuna esecuzione di benchmark: sei compilazioni con `-Xptxas -v` dei tre backend che tengono accumulatori in registro, nelle due precisioni. Si raccolgono registri per thread, stack frame, spill store e load, e shared memory statica, e si selezionano dal log completo i casi $k \in \{3, 8, 20, 32\}$. Il numero di registri è espresso in registri a 32 bit in *entrambe* le precisioni, quindi i due valori sono confrontabili direttamente — ed è questo che rende leggibile l’attesa: a $k = 32$ un accumulatore `double` occupa due registri e uno `float` uno solo, quindi il passaggio a FP32 dovrebbe dimezzare la pressione e, se c’era spill, farlo sparire.

Due avvertenze che la campagna stessa mette per iscritto, e che vanno rispettate nella lettura: `ptxas` non misura la shared memory *dinamica*, quindi per `cuda_warp_smem` il dato va correlato con il piano runtime dei CSV di C7.1 (per i k specializzati, `byte = x_rows_per_tile · (k+1) · s`); e `blocks_per_sm` è una *capacità teorica*, non l’occupancy raggiunta, che richiederebbe un profilo.

8.6 Riproducibilità

Ogni campagna scrive accanto ai CSV un file di metadati con: comando eseguito, timestamp, tutti i parametri del `.conf`, commit git e stato del working tree, versione di `nvcc` e di `mpirun`, informazioni sulla GPU. Un risultato senza questi metadati non è confrontabile con un altro raccolto due settimane dopo.

Tre regole operative discendono dall’organizzazione dei file, e sono documentate nel repository perché violarle costa una campagna:

- **rilanciare una campagna sovrascrive i suoi CSV:** la cartella va archiviata prima di ripetere;
- **la validazione si fa a parte:** nessuna campagna usa `-check`, perché l’oracolo seriale costa $O(MNk)$ sul rank 0 e falserebbe i tempi. Si valida prima, su taglie piccole (capitolo 9);
- **trasporto MPI:** il runner usa `bt1=self,sm` per default; il componente `sm` non esiste in Open MPI 3.x/4.x, dove si chiama `vader`. Se le esecuzioni multi-rank falliscono con un errore di BTL, va verificato con `ompi_info -param bt1 all`.

Capitolo 9

Correttezza e validazione

La traccia chiede il collaudo funzionale contro un'implementazione seriale minimale. Il progetto la usa come oracolo, ma la strategia di correttezza è articolata su quattro livelli, perché i modi di sbagliare sono di natura diversa e un solo controllo non li copre.

9.1 L'oracolo seriale

`src/serial/serial.c` è la trascrizione letterale della eq. (2.1): tre cicli annidati nell'ordine $i \rightarrow c \rightarrow j$, cioè lo schema B. La scelta è deliberata: l'oracolo deve essere *ovviamente* corretto, non veloce, e usare un ordine dei cicli diverso da quello dei backend riduce la probabilità che un errore concettuale sull'ordinamento passi inosservato in entrambi.

Con `-check` il rank 0 raccoglie Y distribuita, ricostruisce da sé A e X globali attraverso il generatore (sezione 2.4) — senza riceverle da nessuno — calcola l'oracolo e confronta. Il costo è $O(MNk)$ seriale, che è proprio il motivo per cui la validazione non fa mai parte di una campagna di misura.

9.2 Una tolleranza che dipende da N

La tolleranza del confronto non è una costante, e non può esserlo: ogni elemento di Y è una riduzione su N termini, e l'errore di arrotondamento di una riduzione dipende da N .

Con dati a media nulla — che è esattamente ciò che il generatore produce, e per questo motivo — gli arrotondamenti hanno segno indipendente e l'errore cresce come $\varepsilon\sqrt{N}$, non come il pessimistico εN del caso in cui tutti i termini abbiano lo stesso segno. La costante del modello è **misurata**, non stimata: confrontando il prodotto distribuito con l'oracolo su $N = 257 \dots 40\,000$, in entrambe le precisioni, l'errore relativo in norma L_2 vale

$$\text{err} \approx 0,17 \cdot \varepsilon \sqrt{N}, \quad (9.1)$$

con un rapporto che resta fra 0,11 e 0,17 su tutto l'intervallo. La soglia adottata è quindi

$$\text{tol}(N) = 8 \cdot \varepsilon \cdot \sqrt{N}, \quad (9.2)$$

cioè un margine uniforme di circa $47\times$ sul modello, su tutte le taglie e in entrambe le precisioni.

Il confronto con una soglia fissa mostra perché la scelta conta, e sbaglia in due modi opposti proprio nell'intervallo di taglie che il progetto misura:

Soglia fissa	Difetto	Conseguenza
<code>double</code> , 10^{-12}	troppo larga	a $N = 257$ l'errore reale è $5,6 \cdot 10^{-16}$: la soglia è ~ 1800 volte l'errore, e a $N = 40\,000$ ancora ~ 140 volte. Un bug che gonfiasse l'errore di due ordini di grandezza passerebbe per PASS.
<code>float</code> , 10^{-5}	si restringe con N	il margine passa da $33\times$ a $N = 257$ a $2,5\times$ a $N = 40\,000$, che è la taglia a cui la campagna lavora. Basta un backend con un altro ordine di riduzione per produrre un FAIL su un risultato corretto — e siccome il binario esce con stato non nullo, quel FAIL abortisce la campagna.

Tabella 9.1: Perché una soglia costante non funziona in nessuna delle due precisioni.

Un ultimo dettaglio che conta: N nella eq. (9.2) è il numero *globale* di colonne di A , non n_{loc} , perché è su tutte e N che l'oracolo somma.

9.3 Il test delle funzioni indice

Da completare con i dati della campagna

La cartella `test/` non è presente nella copia di lavoro corrente, mentre il `Makefile` continua a costruire `bin/test_index` da `test/test_index.c` e la sonda `check-cxx` da `test/cxx_iface.cpp`: allo stato attuale `make`, `make test` e `make check-cxx` non possono funzionare. Ripristinare la cartella dal repository (`git checkout -- test/`) prima della consegna, oppure aggiornare questa sezione se i test sono stati rimossi di proposito.

Le tre funzioni di sezione 4.2 hanno un test isolato (`test/test_index.c`, `make test`) che non richiede MPI. Verifica su molte combinazioni (n, p) , inclusi i casi limite $p > n$ e $n \bmod p \neq 0$, tre proprietà:

- $\sum_i \text{size}(n, p, i) = n$: la partizione copre tutto;
- $\text{start}(n, p, i+1) = \text{start}(n, p, i) + \text{size}(n, p, i)$: le parti sono contigue e non si sovrappongono;
- `owner` è l'inversa di `start/size`: le due direzioni sono coerenti.

Sono i controlli che intercettano l'errore di un solo elemento, cioè proprio quello che si propagherebbe in silenzio in tutto il resto del sistema.

9.4 La matrice di collaudo

Tabella 9.2: I controlli eseguiti e che cosa ciascuno intercetta.

Comando	Che cosa protegge
<code>make test</code>	partizionamento degli indici, senza MPI
<code>make check-cxx</code>	<code>kernel.h</code> e <code>util.h</code> restano includibili da C++ e tutti i simboli dell'interfaccia conservano il linkage C (verificato con <code>nm</code>): intercetta le regressioni di <code>extern "C"</code> e <code>restrict</code> su qualunque macchina, anche senza <code>nvcc</code>
<code>make check</code>	prodotto distribuito contro oracolo seriale, doppia precisione
<code>make PREC=float check</code>	stessa validazione in singola precisione: è il caso in cui una tolleranza sbagliata si manifesta
<code>make check-padding</code>	<code>lda = n_{loc} + 8</code> : verifica che tutti i backend onorino la leading dimension di A
<code>make KERNEL=<nome> check</code>	la stessa validazione su ciascun backend GPU, che ha un ordine di riduzione diverso da quello di CPU
<code>make -B KERNEL=cuda_warp EXTRA_NVCCFLAGS='-Xptxas -v'</code>	ispezione dell'uso dei registri per warp: non è correttezza ma è il controllo che dice se la specializzazione su k sta facendo ciò che si crede

La validazione va eseguita su ogni combinazione di backend e precisione *prima* della campagna corrispondente, su taglie piccole. Un backend validato solo in doppia precisione non è validato: l'ordine della riduzione cambia con il kernel, e in singola precisione l'effetto è misurabile.

Da completare con i dati della campagna

Riportare l'esito effettivo dei controlli sul server, con il valore di `rel_err` misurato e la soglia corrispondente, per almeno: backend di CPU in `double` e `float`; `cuda_warp` e `cuda_warp_smem` in entrambe le precisioni; `cublas`; il caso con padding. Una tabella con colonne (backend, precisione, N , `rel_err`, `tol`, esito) è la forma più compatta.

Tabella 9.3: Esiti della validazione. **[DA FARE: compilare dopo l'esecuzione sul server]**

Backend	Precisione	N	<code>rel_err</code>	<code>tol</code>	Esito
<code>scheme_a</code>	double	—	—	—	—
<code>scheme_a</code>	float	—	—	—	—
<code>scheme_a</code> (padding)	double	—	—	—	—
<code>cuda_naive</code>	double	—	—	—	—
<code>cuda_warp</code>	double	—	—	—	—
<code>cuda_warp</code>	float	—	—	—	—
<code>cuda_warp_smem</code>	double	—	—	—	—
<code>cuda_warp_smem</code>	float	—	—	—	—
<code>cuda_warp_tiled</code>	double	—	—	—	—
<code>cublas</code>	double	—	—	—	—

Capitolo 10

Risultati

10.1 Stato della raccolta dati

Tabella 10.1: Stato delle campagne al momento di questa versione della relazione.

Campagna	Stato	Dati disponibili	Sezione
C3 (baseline CPU)	parziale	build specializzata, $P = 1, 10000^2$	sezione 10.2
C3 (build generica)	da eseguire	—	sezione 10.2.3
C2 (k -sweep, 6 backend)	da eseguire	—	sezione 10.3
C1 (taglia $\times$ rapporto)	da eseguire	—	sezione 10.4
C6 (forma della griglia)	da eseguire	—	sezione 10.5
C4 (strong scaling)	da eseguire	—	sezione 10.6
C5 (weak scaling)	da eseguire	—	sezione 10.7
C7.1 (precisione $\times k$)	da eseguire	—	sezione 10.8
C7.2 (risorse per precisione)	da eseguire	—	sezione 10.9

Da completare con i dati della campagna

Tempo macchina stimato per completare la raccolta: circa un'ora, escluse le compilazioni, nell'ordine C3 (build generica) $\rightarrow$ C2 $\rightarrow$ C1 $\rightarrow$ C6 $\rightarrow$ C4 $\rightarrow$ C5 $\rightarrow$ C7. Le sezioni che seguono sono già strutturate con le tabelle e le figure definitive: resta da sostituire i segnaposto con i numeri.

Va inoltre popolata sezione 10.10: i tempi esclusi dalla metrica ufficiale non richiedono nessuna esecuzione aggiuntiva, perché ogni campagna li scrive già in coda al proprio CSV.

10.2 C3 — baseline di CPU

Configurazione: `scheme_a`, doppia precisione, $M = N = 10\,000$, $P = 1$ (griglia 1×1), `-reps 20 -warmup 5`, seed 20 252 026, generazione locale di A e X . Senza comunicazione e senza GPU: è il nucleo di calcolo puro, ed è il denominatore di ogni speedup della relazione.

Tabella 10.2: Campagna C3, build specializzata. t è il tempo ufficiale medio su 20 repetition; le ultime due colonne sono derivate, non misurate, e sono definite nel testo.

k	t [s]	CV [%]	GFLOPS	B_A [GB/s]	B_X [GB/s]
3	0,1336	1,38	4,49	5,99	18,0
6	0,1424	1,17	8,43	5,62	33,7
8	0,1473	1,10	10,87	5,43	43,5
20	0,7680	0,37	5,21	1,04	20,8
32	1,3073	0,43	4,90	0,61	19,6
<i>fallback generico (due passaggi su A, cfr. eq. (5.1))</i>					
40	1,7591	0,46	4,55	0,45	18,2
64	3,0332	0,37	4,22	0,26	16,9

Il coefficiente di variazione resta fra 0,37% e 1,38%: tutte le differenze discusse qui sotto sono di ordini di grandezza superiori alla dispersione delle misure. Il preprocessing del backend ($t_{\text{setup}} \approx 1,4 \mu\text{s}$) è trascurabile, come atteso per un backend che si limita a registrare un puntatore.

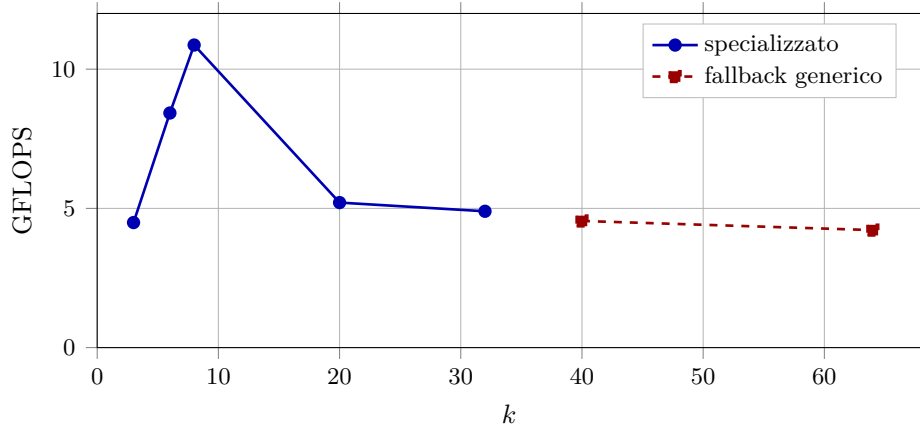


Figura 10.1: C3: prestazioni del nucleo di CPU al variare di k . Il massimo cade a $k = 8$; oltre, il throughput crolla.

10.2.1 Lettura: due regimi separati da una soglia di cache

La curva di fig. 10.1 ha una forma che non si spiega con il solo conteggio delle operazioni: fra $k = 8$ e $k = 20$ il lavoro cresce di $2,5\times$ ma il tempo cresce di $5,2\times$. Il modo più diretto di guardare il fenomeno è normalizzare rispetto al numero di elementi di A elaborati, che è costante:

$$\tau(k) = \frac{t(k)}{MN} \quad [\text{tempo per elemento di } A]. \quad (10.1)$$

Tabella 10.3: Costo per elemento di A e dimensione di X_{loc} .

k	τ [ns]	$\tau/\tau_{k=3}$	$ X_{\text{loc}} $
3	1,34	1,00	0,24 MB
6	1,42	1,07	0,48 MB
8	1,47	1,10	0,64 MB
20	7,68	5,75	1,60 MB
32	13,07	9,78	2,56 MB
40	17,59	13,17	3,20 MB
64	30,33	22,70	5,12 MB

Per $k \in \{3, 6, 8\}$ il costo per elemento di A è **quasi costante**: triplicare il numero di FMA per elemento aumenta il tempo del 10%. È la firma di un regime in cui il collo di bottiglia non è il calcolo ma lo streaming di A : la banda effettiva su A , $B_A = MNs/t$, vale da 5,4 GB/s a 6,0 GB/s, un valore plausibile per un singolo core, e le FMA aggiuntive si nascondono dietro le latenze di memoria.

Da $k = 20$ in poi il comportamento cambia natura. Se il limite fosse la banda su A , τ resterebbe piatto; se fosse il picco di calcolo, τ crescerebbe *proporzionalmente* a k . Non fa né l'una né l'altra cosa: da $k = 8$ a $k = 20$ cresce di $5,2\times$ per $2,5\times$ di lavoro. Il candidato che spiega il fattore è il traffico su X della eq. (5.2). Definendo

$$B_X = \frac{MNks}{t} \quad [\text{banda effettiva sulla riletture di } X], \quad (10.2)$$

la colonna corrispondente di tabella 10.2 mostra un valore sostanzialmente **costante** intorno a da 17 GB/s a 21 GB/s per tutti i $k \geq 20$ — inclusi $k = 40$ e $k = 64$, che passano dal fallback e leggono A due volte. Un plateau di banda su cinque punti che coprono un fattore 3,2 in k è una firma difficile da attribuire al caso.

Ipotesi: oltre $k \approx 8$ la riletture di X esce dalla cache privata

Nello schema A ogni riga di A percorre tutta X_{loc} . Finché X_{loc} sta nella cache privata del core, quelle riletture costano poco e il nucleo è limitato dallo streaming di A ($k \leq 8$: B_A costante, τ piatto). Quando X_{loc} supera quella capacità — fra 0,64 MB a $k = 8$ e 1,6 MB a $k = 20$ — ogni riga di A ricomincia a leggere X dal livello successivo della gerarchia, e il tempo diventa proporzionale a $MNks$ diviso la banda di quel livello: da qui il plateau di B_X .

Il valore del plateau, da 17 GB/s a 21 GB/s, è coerente con una banda di cache condivisa su singolo core, e nettamente superiore alla banda DRAM per core che si legge nel regime $k \leq 8$ (~ 6 GB/s): non si tratta quindi di traffico verso la memoria centrale.

Come falsificarla. (i) Leggere la capacità della cache privata con `lscpu -C` e verificare che cada fra 0,64 e 1,6 MB; (ii) contare i miss del livello sospetto con `perf stat -e` sui k ai due lati della soglia; (iii) ripetere C3 a N dimezzato: se l'ipotesi è giusta, la soglia in k deve raddoppiare, perché $|X_{\text{loc}}| = Nks$.

L'ipotesi alternativa — spill degli accumulatori oltre il numero di registri vettoriali disponibili — è compatibile con il salto fra $k = 8$ e $k = 20$, ma non spiega perché B_X resti costante fino a $k = 64$ né perché il fallback a $KB = 32$, che tiene al massimo 32 accumulatori indipendentemente da k , mostri lo stesso plateau. Le due cause non si escludono, e il terzo controllo elencato sopra le separa in modo netto. È la verifica da eseguire prima della consegna.

10.2.2 Conseguenza pratica

Se l'ipotesi regge, il rimedio è noto e non richiede di cambiare schema: basta bloccare il ciclo su j in fette di X che stiano in cache, riusando ogni fetta su tutte le righe di A prima di passare alla successiva — lo stesso principio che `cuda_warp_smem` applica sulla GPU con la shared memory (sezione 6.4). È il primo dei lavori futuri (sezione 11.3), ed è anche il motivo per cui vale la pena identificare la causa e non limitarsi a osservare che “oltre $k = 8$ va peggio”.

Figura da generare

GFLOPS in funzione di k , una curva per backend, scala logaritmica sull'asse y . Sovrapporre come linea orizzontale il tetto FP64 di tabella 7.2 (350 GFLOP/s): la distanza da quel tetto è il risultato, non il numero assoluto.

Figura 10.2: C2: confronto fra backend al variare di k .

Le tre domande a cui questa campagna deve rispondere, e che vanno affrontate esplicitamente nel commento ai dati:

1. **Quanto vale il passaggio da `cuda_naive` a `cuda_warp`?** È la misura del recupero del riuso di A in registro e della coalescenza, cioè del salto di intensità aritmetica della eq. (2.3). Ci si attende un guadagno crescente con k , perché è k il fattore di riuso perduto dalla baseline.
2. **La shared memory guadagna qualcosa?** Il roofline di sezione 7.1 prevede di no, per $k \geq 6$ in FP64: se la misura lo conferma, la conferma è il risultato. Il controllo incrociato è ripetere il confronto in FP32, dove il modello prevede un regime memory-bound e quindi un guadagno.
3. **Dove sta il crossover fra `cuda_warp_smem` e `cuda_warp_tiled`?** Il primo tiene k accumulatori per lane, il secondo ne tiene `WARP_COL_TILE` al prezzo di rileggere A : il rapporto fra i due tempi in funzione di k individua il punto in cui la pressione sui registri costa più della rilettura.

10.3.1 Sweep sul padding della shared memory

Da completare con i dati della campagna

Confronto `cuda_warp_smem` con `SMEM_PAD=1` contro `SMEM_PAD=0` a $k = 32$, dove l'analisi di sezione 6.4.1 prevede il conflitto a 32 vie. È una previsione puntuale e falsificabile: il padding deve mostrare un guadagno a $k = 32$ e nessun effetto apprezzabile a $k = 3$.

k	SMEM_PAD=0	SMEM_PAD=1	guadagno
3	—	—	—
8	—	—	—
32	—	—	— (atteso: il più alto)

Tabella 10.6: Effetto del padding sul conflitto di banchi. **[DA FARE: eseguire]**

10.3.2 Sweep sulla dimensione del blocco

Da completare con i dati della campagna

`BLOCK` $\in \{64, 128, 192, 256, 384, 512, 1024\}$ su `cuda_warp`, a k fisso. Serve a documentare che il default 256 non è stato scelto solo per argomento teorico (divisore di 1024, quattro blocchi per SM) ma anche misurato. I valori 192 e 384 sono i controlli: se l'argomento sull'occupancy è corretto, devono essere peggiori dei divisori di 1024.

10.4 C1 — taglia e rapporto $M:N$

Da completare con i dati della campagna

Suite `c1_size_ratio_cpu.txt` e `c1_size_ratio_gpu.txt`, i dodici punti di tabella 8.4. È la campagna esplicitamente richiesta dalla traccia e non può mancare nella consegna.

Tabella 10.7: C1: GFLOPS per passo e rapporto. Una tabella per backend.

Passo	CPU (<code>scheme_a</code>)			GPU		
	1:1	3:1	1:2	1:1	3:1	1:2
s1	—	—	—	—	—	—
s2	—	—	—	—	—	—
s3	—	—	—	—	—	—
s4	—	—	—	—	—	—

Figura da generare

GFLOPS in funzione di MN (asse x logaritmico), tre curve per i tre rapporti, un pannello per CPU e uno per GPU. Poiché il lavoro è uguale a parità di passo, ogni scostamento verticale fra le tre curve è effetto della *forma*.

Figura 10.3: C1: effetto della taglia e del rapporto $M:N$.

Da discutere nel commento: la crescita con la taglia (l'ammortamento dei costi fissi, e su GPU il raggiungimento di parallelismo sufficiente a saturare gli SM) e, separatamente, la distanza fra le tre curve, che è l'effetto della forma previsto in sezione 4.6 e in sezione 7.1 — con l'attesa che il rapporto 3:1 (M grande, molti warp) sia favorevole su GPU e che 1:2 lo sia meno.

10.5 C6 — perché la griglia è bidimensionale

Da completare con i dati della campagna

Suite `c6_grid_shape.txt`, tutte le fattorizzazioni di $P = 8, 16, 20$. È la campagna che giustifica sperimentalmente il requisito centrale della traccia; le forme $1 \times P$ e $P \times 1$ sono le due decomposizioni monodimensionali e vanno sullo stesso grafico.

Tabella 10.8: C6: $P = 20$, tutte le forme ordinate.

Forma $P_r \times P_c$	t_{bcast}	t_{reduce}	t_{total}	GFLOPS
1×20 (1D)	—	—	—	—
2×10	—	—	—	—
4×5	—	—	—	—
5×4	—	—	—	—
10×2	—	—	—	—
20×1 (1D)	—	—	—	—

Figura da generare

Due assi y sovrapposti: t_{bcast} e t_{reduce} lungo la sequenza di forme. La eq. (4.4) prevede che si muovano in direzioni *opposte* e che il minimo del totale cada in mezzo, non agli estremi.

Figura 10.4: C6: broadcast e reduce in funzione della forma della griglia.

Questa è la verifica più diretta del modello di costo: la eq. (4.5) predice, per $M = N$ e $P = 20$, un ottimo vicino a 4×5 o 5×4 , e un peggioramento marcato agli estremi. Se i dati lo confermano, la scelta di `grid_default_shape` risulta giustificata da una misura e non da un'intuizione; se non lo confermano, il modello va corretto, ed è comunque un risultato da riportare.

10.6 C4 — strong scaling

Da completare con i dati della campagna

Suite `c4_strong_scaling.txt`: $20\,000 \times 20\,000$, $P \in \{1, 2, 4, 8, 16, 20\}$, $k \in \{3, 32\}$.

Tabella 10.9: C4: speedup ed efficienza a problema fisso.

P	t	$k = 3$				$k = 32$			
		speedup	eff.	% comm.		speedup	eff.	% comm.	
1	—	1,00	1,00	—	—	1,00	1,00	—	—
2	—	—	—	—	—	—	—	—	—
4	—	—	—	—	—	—	—	—	—
8	—	—	—	—	—	—	—	—	—
16	—	—	—	—	—	—	—	—	—
20	—	—	—	—	—	—	—	—	—

Figura da generare

A sinistra: speedup contro P , con la retta ideale come riferimento, due curve ($k = 3$ e $k = 32$). A destra: scomposizione di t_{total} in t_{bcast} , t_{local} , t_{reduce} come frazioni impilate.

Figura 10.5: C4: strong scaling e composizione del tempo.

L'attesa, dalla eq. (4.3): con $k = 3$ la comunicazione è una frazione piccola e la scalabilità resta vicina all'ideale; con $k = 32$ il volume comunicato è dieci volte tanto ed è lì che l'efficienza si degrada. Il pannello di destra deve rendere visibile *quale* delle due collettive domina, e il confronto con la legge di Amdahl applicata alla frazione non parallelizzabile misurata chiude l'analisi.

10.7 C5 — weak scaling

Da completare con i dati della campagna

Suite `c5_weak_scaling.txt`: blocco locale costante 5000×5000 , $M = 5000P_r$, $N = 5000P_c$.

Tabella 10.10: C5: tempo a lavoro per processo costante.

P	Griglia	$M \times N$	t_{total}	t_{comm}	efficienza
1	1×1	5000×5000	—	—	1,00
2	—	—	—	—	—
4	—	—	—	—	—
8	—	—	—	—	—
16	—	—	—	—	—
20	—	—	—	—	—

Poiché il lavoro per rank è costante, il tempo ideale è piatto e tutto ciò che sale è comunicazione pura: è la misura più pulita del costo delle collettive, e il confronto con la previsione della eq. (4.3) permette di stimare α e β per interpolazione. Va ricordato nel commento che su

una griglia 2D il weak scaling cambia anche il rapporto $M:N$ (sezione 4.6), quindi l'effetto misurato qui non è puramente quello di P .

10.8 C7.1 — quanto rende la singola precisione

Da completare con i dati della campagna

Suite `experiments/c7_1_precision_k.txt`: cinque backend GPU in coppie double/float, $M = N = 10\,000$, $P = 1$, BLOCK=256, WARP_COL_TILE=8. Colonna da leggere: `t_kernel_mean_s`, e da essa lo speedup della eq. (8.6).

Tabella 10.11: C7.1: rapporto $S_{32/64} = t_{\text{kernel}}^{\text{fp64}}/t_{\text{kernel}}^{\text{fp32}}$ per backend e per k .

Backend	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
<code>cuda_naive</code>	—	—	—	—	—
<code>cuda_warp</code>	—	—	—	—	—
<code>cuda_warp_smem</code>	—	—	—	—	—
<code>cuda_warp_tiled</code>	—	—	—	—	—
<code>cublas</code>	—	—	—	—	—
<i>previsione roofline</i>	≈ 2	<i>crescente con k, fino a $\sim 20\times$ a $k = 32$</i>			

È la verifica più diretta del modello di capitolo 7, ed è falsificabile in modo netto. Il modello prevede che in FP64 il tetto sia il calcolo e non dipenda da k , mentre in FP32 sia la banda e cresca linearmente con k : il rapporto fra i due deve quindi **crescere con k** . Uno speedup piatto e vicino a 2 — il rapporto fra le dimensioni degli scalari, cioè ciò che si otterrebbe se entrambe le precisioni fossero memory-bound — falsificherebbe la previsione, e andrebbe riportato come tale.

Figura da generare

$S_{32/64}$ in funzione di k , una curva per backend, con due linee orizzontali di riferimento: $S = 2$ (entrambe memory-bound) e la previsione del roofline. È il grafico che decide se il modello di capitolo 7 regge.

Figura 10.6: C7.1: guadagno della singola precisione al variare di k .

Per `cuda_warp_smem` vanno riportate anche le due colonne del piano, perché cambiano con la precisione e sono metà della spiegazione:

k	x_rows_per_tile		blocks_per_sm	
	double	float	double	float
3	—	—	—	—
8	—	—	—	—
20	—	—	—	—
32	—	—	—	—

Tabella 10.12: C7.1: piano di tiling scelto a runtime nelle due precisioni. [DA FARE: compilare]

10.9 C7.2 — perché: registri, spill e shared memory

Da completare con i dati della campagna

Suite `experiments/c7_2_precision_resources.txt`: sei compilazioni con `-Xptxas -v`, nessun benchmark. Estrarre dal log completo `*_ptxas*.txt` i casi $k \in \{3, 8, 20, 32\}$.

Tabella 10.13: C7.2: risorse per thread dichiarate da `ptxas` (registri a 32 bit in entrambe le precisioni).

Backend	Prec.	k	reg./thread	stack	spill st./ld.	shared statica
cuda_warp	double	32	—	—	—	—
cuda_warp	float	32	—	—	—	—
cuda_warp_smem	double	32	—	—	—	—
cuda_warp_smem	float	32	—	—	—	—
cuda_warp_tiled	double	32	—	—	—	—
cuda_warp_tiled	float	32	—	—	—	—
<i>ripetere per $k = 3, 8, 20$</i>						

L'attesa è leggibile in anticipo, ed è ciò che rende la tabella un esperimento e non un inventario: a $k = 32$ un accumulatore `double` occupa due registri a 32 bit e uno `float` uno solo, quindi il passaggio a FP32 dovrebbe dimezzare la pressione sui registri e, se a $k = 32$ in doppia precisione compare dello spill, farlo sparire. Se lo spill c'è, spiega *da solo* una parte dello speedup di tabella 10.11 che il roofline non prevedeva — ed è il motivo per cui le due metà della campagna vanno lette insieme.

Due limiti da rispettare nella lettura, che la campagna stessa dichiara: `ptxas` riporta la shared memory *statica* e non quella dinamica, quindi per `cuda_warp_smem` il dato va ricostruito dal piano runtime di tabella 10.12 come `x_rows_per_tile · (k + 1) · s` byte; e `blocks_per_sm` è una capacità teorica dichiarata dall'API dell'occupancy, non l'occupancy raggiunta, che richiederebbe un profilo con `ncu`.

10.10 Il costo dei dati: trasferimenti e preprocessing

Da completare con i dati della campagna

Nessuna esecuzione aggiuntiva: le colonne di tabella 8.2 sono già in coda al CSV di ogni campagna. Compilare le due tabelle seguenti estraendole dalle righe GPU di C2 (per la dipendenza da k) e di C1 (per la dipendenza dalla taglia).

Questa sezione risponde a una domanda che la traccia permette di non mettere in T ma non di ignorare: con $k \leq 32$, quanto costa portare i dati sulla scheda rispetto a calcolarci sopra?

Tabella 10.14: Scomposizione della fase locale su GPU, secondo la eq. (6.1). Percentuali di t_{local} .

k	$t_{\text{H2D } X}$	t_{kernel}	$t_{\text{D2H } Y}$	t_{launch}	$t_{\text{local}} [\text{s}]$
3	—	—	—	—	—
8	—	—	—	—	—
32	—	—	—	—	—

Da discutere: i due trasferimenti per invocazione scalano con $n_{\text{loc}}k$ e $m_{\text{loc}}k$, mentre l’overhead del runtime non scala affatto — quindi il loro peso relativo deve muoversi in direzioni opposte al crescere della taglia, ed è esattamente ciò che la scomposizione permette di verificare invece di asserire. Se a $k = 3$ i trasferimenti pesano quanto il kernel o più, quello è il risultato: dice che a k piccolo il problema non è il calcolo, ed è l’argomento più forte per un MPI CUDA-aware (sezione 11.3).

Tabella 10.15: Preprocessing e banda dei trasferimenti, per una configurazione rappresentativa.

Voce	Valore	Nota
t_{init}	—	costo fisso, indipendente dalla taglia
t_{setup}	—	cresce con la taglia
t_{alloc}	—	il trasferimento tolto dal cammino misurato
$t_{\text{setup}}^{\text{H2D } A}$	—	da confrontare con il picco del bus
bw_h2d_A_gbs	—	messaggi piccoli: attesa banda molto inferiore
$\text{bw_h2d_X_gbs} / \text{bw_d2h_Y_gbs}$	—	generazione, esclusa da T
$t_{\text{prep}}^{\text{gen } A}$	—	0 con <code>-a-mode local</code>
$t_{\text{distrib } A}^{\text{prep}}$	—	0 con <code>X_LAYOUT=row</code>
$t_{\text{convert } X}^{\text{prep}}$	—	

Il confronto fra bw_h2d_A_gbs e le due bande per invocazione è istruttivo di per sé: A è un unico trasferimento da gigabyte e dovrebbe avvicinarsi al picco del bus, mentre X e Y sono messaggi da qualche decina o centinaio di kilobyte, dominati dalla latenza. È la stessa distinzione fra α e β della eq. (4.3), su un altro collegamento.

10.11 Sintesi

Da completare con i dati della campagna

Completare a raccolta ultimata con: la tabella riassuntiva (backend $\times k$, migliore configurazione, GFLOPS, % del tetto roofline pertinente); il confronto fra la previsione di sezione 7.1 e la misura, con lo scarto; l’esito dei tre punti elencati in sezione 10.3.

Capitolo 11

Conclusioni

11.1 Che cosa è stato realizzato

Il progetto implementa il prodotto $Y \leftarrow AX$ per matrice densa e multivettore stretto, distribuito su una griglia di processi bidimensionale con MPI e accelerato all'interno del singolo processo con CUDA. Il codice è in C11 per la parte host e distribuita, in CUDA C++ per i kernel di device, e copre tutti i requisiti della traccia elencati in ??.

Le scelte che hanno avuto l'effetto maggiore, e che sono anche quelle su cui poggia l'analisi:

- **lo schema A** ($i \rightarrow j \rightarrow c$), che legge ogni elemento di A una volta sola e lo riusa nei k accumulatori in registro, portando l'intensità aritmetica da $2/s$ a $2k/s$ FLOP/B — il massimo ottenibile leggendo A una volta;
- **la griglia 2D**, che riduce il volume comunicato da $\Theta(kM)$ o $\Theta(kN)$ delle decomposizioni monodimensionali a $\Theta(k\sqrt{MN}/\sqrt{P})$;
- **il ciclo di vita a tre fasi** del kernel locale, che sposta fuori dal cronometro tutto ciò che riguarda A — inclusa la copia H2D, che altrimenti misurerebbe il PCIe invece della GPU — rendendo la prescrizione della traccia una proprietà strutturale e non una sottrazione a posteriori;
- **una variabile per volta** fra le varianti di kernel, che è la condizione perché ogni differenza misurata sia attribuibile;
- **la strumentazione completa dei tempi esclusi** (sezione 8.3): la traccia permette di non metterli in T , e il progetto li misura comunque, scomposti per fase. La fase locale su GPU è separata nei suoi quattro addendi (eq. (6.1)), il preprocessing del backend nelle sue tre voci, e i trasferimenti riportano anche i byte, così che un tempo di copia si possa giudicare e non solo constatare.

11.2 Che cosa dicono i dati finora

Da completare con i dati della campagna

Questa sezione va riscritta a raccolta ultimata. Al momento l'unica campagna disponibile è C3, e il suo risultato principale è riportato qui sotto.

Il nucleo di CPU raggiunge 10,9 GFLOP/s a $k = 8$ su $10\,000 \times 10\,000$ in doppia precisione a rank singolo, e presenta una discontinuità netta fra $k = 8$ e $k = 20$: il costo per elemento di A passa da 1,47 ns a 7,68 ns, mentre il lavoro cresce solo di $2,5\times$. L'analisi di sezione 10.2 attribuisce il fenomeno alla rilettura di X , che oltre una certa soglia esce dalla cache privata del core, e propone tre controlli per falsificare l'ipotesi. La firma a sostegno è un plateau

di banda effettiva su X intorno a da 17 GB/s a 21 GB/s, costante su cinque valori di k che coprono un fattore 3,2.

Il modello roofline prevede inoltre che sulla GPU, in doppia precisione, il nucleo sia compute-bound per ogni $k \geq 6$, con tetto a 350 GFLOP/s indipendente da k , mentre in singola precisione resti memory-bound per tutti i k della traccia con tetto proporzionale a k . È una previsione forte e falsificabile, e le campagne C1, C2 e soprattutto C7 la mettono alla prova: quest'ultima ne isola la conseguenza più netta — il guadagno della singola precisione deve crescere con k — e la accompagna con i dati di `ptxas` che dicono se una parte del guadagno venga invece dalla pressione sui registri.

11.3 Lavori futuri

Blocking del ciclo su j sul nucleo di CPU. È la conseguenza diretta dell'analisi di sezione 10.2: percorrere X a fette che stiano nella cache privata, riusando ogni fetta su tutte le righe di A prima di passare alla successiva. Non cambia lo schema, cambia solo l'ordine di visita, ed è lo stesso principio che `cuda_warp_smem` applica sulla GPU con la shared memory. Se l'ipotesi sulla soglia di cache regge, è l'intervento con il rapporto guadagno/costo migliore dell'intero progetto per $k \geq 20$.

Parallelizzazione OpenMP interna al processo. La traccia ammette OpenMP *oppure* CUDA e il progetto ha scelto CUDA. Un backend OpenMP dietro la stessa interfaccia `local_gemm` permetterebbe di usare i circa venti core del nodo anche quando la GPU è occupata, e di misurare il regime ibrido MPI+OpenMP con lo stesso cronometro degli altri.

MPI CUDA-aware. L'interfaccia `local_gemm` è stata progettata agnostica rispetto alla posizione della memoria proprio per consentirlo: passando puntatori device direttamente a `MPI_Bcast` e `MPI_Reduce` si eliminerebbero H2D di X e D2H di Y dal cammino di ogni invocazione, senza riscrivere né `mpi_matmul` né i kernel. Il primo passo è verificare il supporto sul nodo di calcolo con `mpi_info | grep cuda`; il secondo è già fatto, perché la scomposizione di sezione 10.10 dice esattamente quanto ci sarebbe da guadagnare — $t_{H2D} X$ e $t_{D2H} Y$ misurati sono il tetto di quel guadagno, e senza quei due numeri l'intervento sarebbe una scommessa.

MPS per la condivisione della GPU. Con MPS attivo i contesti di più rank non si alternerebbero sulla scheda (sezione 6.7), e i numeri multi-rank su GPU tornerebbero interpretabili come tempo di kernel invece che come throughput aggregato. È anche il modo per replicare, con analisi esplicita della serializzazione, i benchmark multi-griglia su GPU — un punto che i lavori di confronto lasciano scoperto.

Profilazione con `ncu`. Le ipotesi formulate nei capitoli precedenti hanno tutte un contatore hardware che le decide: occupancy e banda L2 per `cuda_warp_smem`, conflitti di banco per lo sweep su `SMEM_PAD`, registri per warp per `cuda_warp_tiled`, kernel interno selezionato per `cublas`. È il passo che trasforma le spiegazioni proposte in spiegazioni verificate.

Appendice A

Interfaccia a riga di comando e formato dei dati

A.1 Opzioni del driver

Opzione	Default	Significato
-M <int>	1024	righe di A
-N <int>	1024	colonne di A (e righe di X)
-k <int>	8	colonne del multivettore
-pr <int>	più quadrata	righe della griglia di processi
-pc <int>	più quadrata	colonne della griglia di processi
-reps <int>	10	repetition cronometrate
-warmup <int>	2	repetition di riscaldamento, non cronometrate
-seed <u64>	fisso	seme del generatore
-a-mode <mode>	local	local: ogni processo genera il proprio blocco; global: il rank 0 materializza A e la distribuisce
-x-mode <mode>	local	come sopra per X , sulla sola riga 0 della griglia
-check	off	valida contro il riferimento seriale
-csv	off	stampa una riga CSV invece del report leggibile
-csv-header	—	stampa l'intestazione CSV ed esce
-csv-raw-file <path>	—	appende una riga per repetition a un CSV “raw”
-h, -help	—	elenco delle opzioni

Tabella A.1: Opzioni di bin/matmul_mpi.

Le quattro combinazioni di `-a-mode` e `-x-mode` sono tutte valide e, a parità di seed, producono gli stessi elementi globali.

`-help` stampa anche il layout di X della build corrente (`X_LAYOUT=row|column`): non è un'opzione di riga di comando, ma è la prima cosa da verificare quando due binari danno tempi diversi sullo stesso problema.

A.2 Colonne del CSV

L'intestazione è emessa dal binario stesso (`-csv-header`), quindi non può divergere dalle colonne effettivamente scritte. È la ragione per cui tutti i CSV di una campagna si possono concatenare senza rischio.

Gruppo di colonne	Contenuto
kernel, scalar, a_mode, x_mode	identificazione della build e della modalità; kernel include i parametri che cambiano il codice generato
M, N, k, P, pr, pc, reps	configurazione del problema
t_bcast_*, t_local_*, t_reduce_*	tempi delle tre fasi, media e deviazione standard
t_total_*	tempo end-to-end reale: media, mediana, minimo, deviazione, CV%
t_official_*	metrica ufficiale della eq. (8.1), stesse statistiche
t_kernel_*	solo kernel, misurato con cudaEvent ; vale -1 sui backend di CPU
t_transfer_runtime_overhead_*	$t_{\text{local}} - t_{\text{kernel}}$: trasferimenti e overhead del runtime host
t_setup_s	preprocessing del backend, fuori da tutte le metriche
gflops, gflops_compute, gflops_kernel	le tre metriche di sezione 8.2
blocks_per_sm, x_rows_per_tile	piano di esecuzione scelto a runtime dal backend; -1 dove il concetto non esiste (sezione 6.4.2)
rel_err	errore relativo contro l'oracolo, oppure -1 senza -check
<i>da qui in poi: voci ESCLUSE dalla metrica ufficiale (sezione 8.3), in coda così che le colonne storiche non si spostino</i>	
t_setup_device_init_s, t_setup_device_alloc_s, t_setup_h2d_A_s, bw_h2d_A_gbs	scomposizione del preprocessing del backend e banda della copia di A
t_h2d_X_*, bw_h2d_X_gbs	H2D di X per invocazione
t_d2h_Y_*, bw_d2h_Y_gbs	D2H di Y per invocazione
t_launch_overhead_*	residuo della eq. (6.1)
t_prep_*	le sei fasi del preprocessing del driver e il loro totale
t_io_csv_raw_s	unico I/O del driver
x_layout	row o column (sezione 6.3.1)
t_prep_convert_X_s	costo della conversione di layout, 0 in row

Tabella A.2: Struttura dell'intestazione CSV.

Il CSV *raw* ha un'altra intestazione — una riga per repetition, senza statistiche, con **t_h2d_X_s**, **t_d2h_Y_s**, **t_launch_overhead_s** e **x_layout** — e va unito separatamente da quello aggregato. Il driver rifiuta di appendere a un file la cui intestazione non coincide con la propria: un CSV misto fra due build sarebbe ambiguo senza dare alcun errore.

A.3 Opzioni di build

Alcune scelte non sono opzioni di riga di comando ma di *compilazione*, perché cambiano il codice generato: metterle a runtime introdurrebbe rami nel percorso caldo e renderebbe impossibile sapere, guardando un binario, che cosa stia misurando. Ognuna entra nel nome del binario e in **kernel_name()** (??).

Variabile	Default	Effetto
KERNEL	scheme_a	implementazione di <code>local_gemm</code> ; <code>.cu</code> ha la precedenza su <code>.c</code> e attiva <code>nvcc</code>
PREC	double	tipo scalare (<code>-DUSE_FLOAT</code>)
FORCE_GENERIC_K	0	disattiva le specializzazioni su k di <code>scheme_a</code>
TEST_A_PADDING	0	impone <code>lda = n_{loc} + n</code>
BLOCK	256	thread per blocco dei backend CUDA
SMEM_PAD	1	scalari di padding per riga del tile (sezione 6.4.1)
TILE_GRANULARITY	32	arrotondamento delle righe per tile
SMEM_BUDGET_BYTES	derivato	forza il budget di shared memory invece di derivarlo dall'occupancy
WARP_COL_TILE	8	colonne per warp in <code>cuda_warp_tiled</code>
X_LAYOUT	row	<code>column</code> rende X compatta per colonne; solo <code>cuda_warp</code> (sezione 6.3.1)
NVCC_ARCH	sm_75	architettura di destinazione

Tabella A.3: Variabili del Makefile.

Appendice B

Riproduzione dei risultati

B.1 Build

```
make                                # bin/matmul_mpi (double, scheme_a)
make FORCE_GENERIC_K=1              # bin/matmul_mpi-generic
make PREC=float                    # bin/matmul_mpi-float
make KERNEL=cuda_naive             # richiede nvcc: solo sul server
make KERNEL=cuda_warp
make KERNEL=cuda_warp X_LAYOUT=column # X compatta per colonne (solo cuda_warp)
make KERNEL=cuda_warp_smem
make KERNEL=cuda_warp_smem SMEM_PAD=0
make KERNEL=cuda_warp_smem TILE_GRANULARITY=8
make KERNEL=cuda_warp_smem SMEM_BUDGET_BYTES=16384
make KERNEL=cuda_warp_tiled
make KERNEL=cuda_warp_tiled WARP_COL_TILE=4
make KERNEL=cublas                 # aggiunge -lcublas da solo
```

Listato B.1: Compilazione delle configurazioni usate nelle campagne.

La riga finale di ogni `make` ricorda quale binario ha prodotto e con quali variabili, budget di shared memory incluso (derivato quando non è forzato).

Ogni comando stampa in chiusura quale binario ha prodotto e con quali variabili. Su una macchina priva di `nvcc` le build CUDA si fermano con un messaggio esplicito e tutto il resto continua a funzionare.

B.2 Validazione

Da eseguire su taglie piccole, prima delle campagne e per ogni combinazione di backend e precisione che si intende misurare.

```
make test                          # funzioni indice, senza MPI
make check-cxx                    # kernel.h e util.h includibili da C++
make check                        # prodotto distribuito contro oracolo, double
make PREC=float check             # stessa validazione in singola precisione
make check-padding                # lda = n_loc + 8
make KERNEL=cuda_warp check
make KERNEL=cuda_warp_smem check
make KERNEL=cublas check
```

B.3 Esecuzione singola

```
# CPU, 4 rank su griglia 2x2
mpirun -np 4 --bind-to core --map-by core \
  ./bin/matmul_mpi -M 8000 -N 8000 -k 8 --pr 2 --pc 2 --reps 10

# GPU, rank singolo: la configurazione con cui si caratterizza un kernel
mpirun -np 1 --bind-to core --map-by core ./bin/matmul_mpi-cuda_warp \
  -M 20000 -N 20000 -k 8 --pr 1 --pc 1 --a-mode local --warmup 2 --reps 10
```

Il binding non è facoltativo sulle esecuzioni multi-rank: senza, le misure sono rumore.

B.4 Campagne

```
./script/run_cuda_experiments.sh --suite experiments/c3_cpu_baseline.txt
./script/run_cuda_experiments.sh --suite experiments/c2_k_sweep.txt
./script/run_cuda_experiments.sh --suite experiments/c1_size_ratio_cpu.txt
./script/run_cuda_experiments.sh --suite experiments/c1_size_ratio_gpu.txt
./script/run_cuda_experiments.sh --suite experiments/c6_grid_shape.txt
./script/run_cuda_experiments.sh --suite experiments/c4_strong_scaling.txt
./script/run_cuda_experiments.sh --suite experiments/c5_weak_scaling.txt
./script/run_cuda_experiments.sh --suite experiments/c7_1_precision_k.txt
./script/run_cuda_experiments.sh --suite experiments/c7_2_precision_resources.txt
```

C7.2 non esegue benchmark: ricompila con `-Xptxas -v` e raccoglie i log delle risorse. I casi k vanno selezionati dal log completo `*_ptxas*.txt`, non dal solo `*_registers.txt`, che omette i nomi delle funzioni e quindi non permette di attribuire i registri al k giusto.

Prima di rilanciare una campagna

Rilanciare sovrascrive i CSV della campagna. Archiviare la cartella:

```
mv results/c4_strong_scaling results/c4_strong_scaling.$(date +%F_%H%M)
```

B.5 Unione dei CSV di una campagna

Tutti i CSV aggregati di una cartella condividono l'intestazione:

```
find results/c1_size_ratio_cpu -name '*.csv' \
  ! -name '*_raw.csv' ! -name '*_failures.csv' \
  | sort | xargs awk 'FNR==1 && NR!=1 {next} {print}' \
  > results/c1_size_ratio_cpu/merged.csv
```

I file `*_raw.csv` hanno un'altra intestazione e vanno uniti separatamente.

B.6 Note operative sul server

- **Nodo di calcolo, non frontend.** Il nodo di login non ha accesso diretto alla GPU: `nvcc -version`, `ompi_info | grep cuda`, `deviceQuery` e `nvidia-smi` vanno eseguiti sui nodi di calcolo attraverso lo scheduler, e così tutti i benchmark.

- **Trasporto MPI.** Il runner usa `btl=self,sm` per default. Il componente `sm` non esiste in Open MPI 3.x/4.x, dove si chiama `vader`: se le esecuzioni multi-rank falliscono con un errore di BTL, verificare con `ompi_info -param btl all` e aggiungere `-btl self,vader` sulla riga di comando, che ha la precedenza sul `.conf`.
- **bash ≥ 4.4 .** Il runner espande array potenzialmente vuoti sotto `set -u`: su bash 3.2 (macOS) si ferma con `unbound variable`. Sul server Linux non si presenta.
- **Memoria della GPU.** Un blocco locale $40\,000^2$ in doppia precisione richiede 12,8 GiB contro i 15,5 GiB della scheda: il programma si ferma prima di allocare, indicando quanta memoria serve e quanta è libera.

B.7 Compilazione di questa relazione

```
cd report && ./compila.sh # oppure: latexmk -pdf main.tex
```

Per la versione di consegna, commentare la riga `\todovisibiletrue` in `preambolo.tex`: i marcatori dei dati mancanti spariscono dal PDF e restano nel sorgente.